SENSOR-ROBUST AND PHYSICS-CONSISTENT FAULT LOCALIZATION IN DISTRIBUTION GRIDS USING SPATIO-TEMPORAL GRAPH NEURAL NETWORKS

24CS798 / 24AI798 – DISSERTATION PHASE – I

Submitted by

P K SANGAMESWAR [CB.SC.P2AIE25016]

in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY
in
ARTIFICIAL INTELLIGENCE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
COIMBATORE – 641 112

NOVEMBER 2026

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF COMPUTING, COIMBATORE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled

SENSOR-ROBUST AND PHYSICS-CONSISTENT FAULT LOCALIZATION IN DISTRIBUTION GRIDS USING SPATIO-TEMPORAL GRAPH NEURAL NETWORKS

submitted by **P K SANGAMESWAR [CB.SC.P2AIE25016]**, in partial fulfillment of the requirements for the award of degree of **Master of Technology in Artificial Intelligence**, is a bonafide record of the work carried out under my guidance and supervision at the Department of Computer Science and Engineering, Amrita School of Computing, Coimbatore.

Dr. M. Anbazhagan
Thesis Advisor
Assistant Professor
Department of Computer Science and Engineering

Dr. P. Bagavathi Sivakumar
Chairperson
Department of Computer Science and Engineering

Evaluated on:

.....
Internal Examiner

.....
External Examiner

ACKNOWLEDGEMENT

I would like to express my deep gratitude to our beloved Chancellor Satguru Sri Mata Amritanandamayi Devi for providing a bright academic climate in this university, which has made this entire task appreciable. I would like to thank the management of Amrita Vishwa Vidyapeetham and the entire fraternity of the Department of Computer Science and Engineering, Amrita School of Computing, for providing me with the necessary infrastructure required for the completion of the project. I wish to record my deep sense of gratitude and profound thanks to my research supervisor Dr. M. Anbazhagan, Assistant Professor, Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, for his keen interest, inspiring guidance and constant encouragement with my work during all stages, to bring this thesis into fruition. I extend my sincere thanks and gratitude to all the members of the review panel, for their timely guidance and suggestions for the improvement of my thesis work. I would like to extend my sincere thanks to my family and friends for helping and motivating me throughout the course of this project. Finally, I would like to thank all those who have helped, guided and encouraged me directly or indirectly during the project work. Above all, I extend my heartfelt salutations to my beloved parents and Almighty, for their blessings and grace, for the successful completion of this thesis.

Name: P K Sangameswar
Roll No: CB.SC.P2AIE25016
Signature:

ABSTRACT

Finding where a fault occurred on a distribution feeder, and not only that one occurred, decides how fast supply can be restored. Spatio-temporal graph neural networks locate faults accurately when only a few micro-phasor measurement units are installed, but their evaluations assume that every sensor always reports. This thesis adopts the spatio-temporal framework of Karabulut et al. on the IEEE 123-bus feeder, regenerates its own corpus of 1.65 million windows, and measures what happens when sensors fail or are noisy after deployment. Across five architectures at three training seeds, clean accuracy did not predict robustness: the most accurate model, with a macro-F1 of 0.846, kept only 5.7% of its score when 10 of 25 sensors were lost. Dropout-aware training, which masks sensors during training, raised this retention to 80.3% with no added parameters or inference latency. It outperformed neighbor-mean, learned and physics-based imputation, and an availability-gated architecture only matched it. The remedy does not protect against measurement noise or against the loss of the sensors nearest the fault, and accuracy on unseen fault scenarios is much lower than on unseen loads. The thesis then builds a fixed physical operator from simulated fault signatures, shows that the sensors can separate almost all fault locations, and designs a physics-consistent network that adds one learned parameter. [The comparison of this network with a noise-augmented reference is pending completion of the runs.]

TABLE OF CONTENTS

ACKNOWLEDGEMENT iii

ABSTRACT iv

List of Tables ix

List of Figures x

Abbreviations xi

List of Symbols xii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Fault Localization Under Partial Observability . . . 2
- 1.3 Problem Definition . . . 3
  - 1.3.1 Fault Localization Task . . . 3
  - 1.3.2 Sensor Loss at Inference . . . 4
  - 1.3.3 Measurement Noise and Unseen Fault Conditions . . . 4
- 1.4 Conditions Beyond the Training Distribution . . . 5
- 1.5 Research Gap . . . 6
- 1.6 Research Questions and Objectives . . . 7
- 1.7 Contributions . . . 8
- 1.8 Scope and Assumptions . . . 9
- 1.9 Methodology Overview . . . 10
- 1.10 Organization of the Thesis . . . 10

2 Literature Survey 12
- 2.1 Overview of Approaches . . . 12
- 2.2 Circuit-Based Fault Location . . . 12
- 2.3 Feature-Based Learning Methods . . . 14
- 2.4 Graph Neural Networks for Fault Location . . . 14
  - 2.4.1 Message Passing on the Feeder Graph . . . 14
  - 2.4.2 Graph-Based Fault Location Models . . . 15
- 2.5 Spatio-Temporal Graph Models and the Measured-Only Graph . . . 15
- 2.6 Missing and Noisy Measurements . . . 17
  - 2.6.1 Imputation and Reconstruction . . . 17
  - 2.6.2 Robustness to Sensor Failure . . . 17
  - 2.6.3 Masking During Training . . . 18
- 2.7 Physics-Aware Learning for Power Systems . . . 18
- 2.8 Summary and Research Gap . . . 19

3 Proposed Work 20

3.1 Simulation and Dataset . . . 20
3.1.1 Feeder and Scenario Design . . . 20
3.1.2 Sensors and the Measured-Only Graph . . . 21
3.1.3 Windows, Labels, Splits and Normalization . . . 22
3.1.4 Data Quality and Reference Bounds . . . 23
3.1.5 Scenario-Held-Out Split . . . 23
3.2 Baseline Architectures . . . 23
3.2.1 Shared Template . . . 23
3.2.2 Spatial Encoders . . . 24
3.2.3 Training Configuration . . . 25
3.3 Sensor-Loss Evaluation Protocol . . . 25
3.3.1 Missingness Model . . . 25
3.3.2 Metrics . . . 27
3.3.3 Statistical Procedure . . . 27
3.4 Dropout-Aware Training . . . 28
3.5 SDR-STGNN: Availability-Gated Aggregation . . . 29
3.6 Comparison and Stress-Test Methods . . . 30
3.6.1 Imputation Before Inference . . . 31
3.6.2 Harder Loss Patterns . . . 32
3.6.3 Measurement Noise . . . 32
3.6.4 Classical Baselines . . . 33
3.7 The Physical Operator . . . 33
3.7.1 Signature Table . . . 33
3.7.2 Observation and Residual . . . 34
3.7.3 Identifiability . . . 36
3.7.4 Physics-Based Imputation and Inference-Time Fusion . . . 36
3.8 PC-STGNN: Physics-Consistent Fusion in the Network . . . 37
3.8.1 Architecture . . . 37
3.8.2 Training Objective . . . 37
3.8.3 Noise Augmentation and Collapse Guard . . . 39
3.8.4 Selection Rules, Reference Model and Verification . . . 39
3.9 Verification of the Evaluation Pipeline . . . 40
3.10 Summary . . . 40

4 Results and Inferences **41**
4.1 Baseline Accuracy Under Full Observability . . . 41
4.2 Degradation Under Sensor Loss . . . 42
4.3 Dropout-Aware Training . . . 43
4.4 Comparison of Remedies . . . 44
4.5 Stress Tests . . . 45
4.5.1 Harder Loss Patterns . . . 45
4.5.2 Measurement Noise . . . 47
4.5.3 Unseen Fault Scenarios . . . 47
4.5.4 Classical Baselines and the Detection–Localization Split . . . 48
4.6 Statistical Significance . . . 48
4.7 Computational Cost . . . 49
4.8 The Physical Operator . . . 50
4.8.1 Fidelity of the Signature Table . . . 50

4.8.2 Identifiability 51
4.8.3 Physics-Only Baselines 52
4.8.4 Inference-Time Fusion 52
4.8.5 Screening of Three First-Order Physics Grafts 53
4.9 PC-STGNN 54
4.9.1 Verification 54
4.9.2 Why Noise Augmentation and the Auxiliary Loss Are Needed 54
4.9.3 Selection of the Loss Weights 55
4.9.4 Comparison With the Noise-Augmented Reference 55
4.10 Discussion 56
4.11 Summary 57

5 Conclusion and Future Enhancements **58**
5.1 Summary of the Work 58
5.2 Conclusions 58
5.3 Limitations 60
5.4 Future Enhancements 60

REFERENCES **62**

LIST OF TABLES

1.1 Positioning of the closest prior work. A check mark means that the item is covered, a dash means that it is not studied, and a tilde means that it is covered in part. 6

2.1 Families of fault localization methods, a representative work, and the main limitation for fault localization under partial observability. . . 13

3.1 Configuration of the simulated corpus. 21
3.2 Event-level partition of the corpus. 22
3.3 Training and software configuration shared by all models. 26
3.4 Comparison methods and stress tests of the sensor-loss study. 31
3.5 Axes of the signature table. 34

4.1 Baseline macro-F1 under full observability, mean $\pm$ standard deviation over three seeds. The best value per column is in bold. 42
4.2 Depth diagnostic for RGSAGE-Max, one seed (42), test macro-F1. . 42
4.3 Standard-trained models under sensor loss: macro-F1 at each level k, the retention at $k = 10$, and the relative and absolute AUDC, mean over three seeds. 43
4.4 Effect of dropout-aware training (DA) on all architectures: macro-F1 at $k = 0$ and $k = 10$, relative AUDC and retention at $k = 10$ for standard (Std) and dropout-aware training. 44
4.5 Remedies on the RGSAGE-Max backbone, macro-F1 at each level k, relative AUDC and retention at $k = 10$, mean over three seeds. Imputation is a no-operation at $k = 0$. 45
4.6 Loss patterns on RGSAGE-Max, macro-F1 at four levels and relative AUDC, standard (Std) and dropout-aware (DA) training. 46
4.7 RGSAGE-Max under measurement noise and sensor loss, macro-F1 for standard (Std) and dropout-aware (DA) training. 47
4.8 Paired two-sided t-tests over three seeds on the RGSAGE-Max backbone ($\mathrm{df} = 2$). Δ is the mean seed-wise difference, and p_{Holm} is adjusted over the seven tests. Std is standard training and DA is dropout-aware training. 49
4.9 Checks of the signature table on the training split. 50
4.10 Training-free methods on the validation split, macro-F1 at each level k and the absolute area. 52
4.11 Inference-time fusion on the dropout-aware RGSAGE-Max, seed 42, validation subsample of 30,000 windows. 53
4.12 Screening of three physics grafts, RGSAGE-Max, seed 42, 15 epochs. 54
4.13 Final tuning of the loss weights, seed 42, 15 epochs, validation split. The columns give the best epoch, the fused macro-F1 at $k = 0$ and averaged over k, and the macro-F1 of the network branch alone. . . 55

4.14 Comparison of PC-STGNN with the noise-augmented reference, validation macro-F1, mean over three seeds. The values are not yet available. 56

LIST OF FIGURES

1.1 The IEEE 123-bus feeder and the measured-only graph used in this thesis. 3
1.2 Processing template shared by all models: temporal encoder, spatial GNN, node classifier and soft voting. 3
1.3 Conditions that a deployed monitoring system meets but a clean benchmark does not. Panel (a) is a schematic. Panels (b) and (c) use the simulated corpus: a three-phase fault at bus 101, and noise added with the method of Section 3.6. 6
1.4 Overview of the research workflow of the thesis. 10

2.1 Families of fault localization methods reviewed in this chapter. . . . 13
2.2 Full-topology graph and measured-only graph. 16

3.1 Dropout-aware training. Only the masking of the input differs from standard training. The example graph is a schematic. 29
3.2 Availability-gated aggregation in SDR-STGNN. 30
3.3 Architecture of PC-STGNN. 38

4.1 Degradation curves of the baselines and of the two mitigations. . . . 43
4.2 Effect of noise and sensor loss together on RGSAGE-Max. 48
4.3 Identifiability of the fault location from 25 magnitude-only sensors. 51

ABBREVIATIONS

AUDC	normalized area under the retention curve
DER	distributed energy resource
GCN	graph convolutional network
GNN	graph neural network
GRU	gated recurrent unit
KCL	Kirchhoff's current law
PC-STGNN	physics-consistent spatio-temporal graph neural network
PINN	physics-informed neural network
PSM	physics signature matching
RMS	root mean square
SDR-STGNN	sensor-dropout-robust spatio-temporal graph neural network
STGNN	spatio-temporal graph neural network
SVM	support-vector machine
μ**PMU**	micro-phasor measurement unit

List of Symbols

a_i	Availability indicator of node i
AGG	Permutation-invariant neighbor aggregation
$\mathcal{A}$	Set of available sensor-phase channels
$\mathcal{D}$	Set of dropped sensors
$D(f_1, f_2; \mathcal{A})$	Separability of two fault locations
F	Number of channels per sample (six)
f_θ	Fault localization model with parameters θ
$\mathcal{G}$	Measured-only graph of the sensor buses
H	Number of attention heads
$h_v^{(l)}$	Representation of node v after l layers
$\mathcal{K}$	Set of dropout levels used in training
k	Number of dropped sensors, $k = \lvert\mathcal{D}\rvert$
$m_{v,p}$	Availability mask of phase p at node v
N	Number of instrumented buses (25)
$\mathcal{N}(v)$	Set of neighbors of node v
N_{hop}	Number of message-passing layers (four)
$P(f\lvert X)$	Normalized soft-vote distribution over the classes
Pr_c, Re_c	Precision and recall of class c
R_{F}	Fault resistance, Ω
$\mathrm{Ret}(k)$	Retention at dropout level k
r_f	Raw residual of location f
$\mathcal{S}$, $\mathcal{S}_0$	Signature tables of faulted and healthy feeder, p.u.
S	Number of samples per window (20)
s_{ip}	Observed voltage of phase p at sensor i, p.u.
$V_{v,s,p}$	RMS voltage magnitude of phase p at node v and sample s, p.u.

W	Learnable layer weights
X	Input window of measurements
y, $\hat{y}$	True and predicted class of a window
z, z'	Node logits before and after fusion
α_{ji}	Attention coefficient from node i to node j
β	Learned weight of the physics fusion
γ	Weight of the auxiliary loss on the network-only logits
ε	Measurement noise sample, p.u.
η	Global load multiplier
θ	Learnable parameters of the model
$\phi^{(l)}$	Learnable node update of layer l
λ_1, λ_2	Weights of the two physical loss terms
ρ_f	Shifted and halved residual of location f
σ	Standard deviation of the measurement noise, p.u.
σ_0	Level of the injected noise in the residual, p.u.
σ_ε	Residual floor, p.u.
τ	Fault type

Chapter 1

INTRODUCTION

This chapter introduces the problem that the thesis addresses. It explains why locating faults on a distribution feeder matters, why the task is hard when only a few sensors are installed, and which gap in the existing work the thesis fills. It then states the objectives, the contributions, the scope of the study and the structure of the remaining chapters.

1.1 Background and Motivation

Electric power distribution networks carry energy over the last stretch between the substation and the consumer. A fault on a feeder, for example a short circuit between a phase conductor and the ground, interrupts supply to every customer downstream of the affected section until that section is isolated and the rest of the feeder is re-energized. A protective relay detects a fault within milliseconds. Locating the fault takes longer. Crews must find the damaged section along a branching network, and an automated restoration scheme can re-energize healthy sections only when it knows which section to isolate. How precisely a monitoring system identifies where a fault occurred, and not merely that one occurred, therefore determines how much of the network can be restored before crews arrive.

Two trends raise the value of accurate localization. Feeders absorb a growing share of distributed energy resources (DERs), and utilities adopt self-healing schemes that reconfigure the network automatically after a fault. Both developments depend on a fast and reliable estimate of the fault location.

Classical localization methods estimate the position of the fault from circuit physics. Impedance-based methods convert the apparent impedance seen from a measurement point into a distance along the feeder (Krishnathevar and Ngu, 2012). Traveling-wave methods time the arrival of the wavefront that the fault launches (Liu et al., 2023). Both families need a measurement coverage that is dense enough to observe the relevant impedances or wavefronts. Distribution grids rarely offer such coverage, because

micro-phasor measurement units (μPMUs) are expensive and need communication infrastructure. A feeder with hundreds of buses may carry only a handful of them. A localization method for distribution grids must therefore work under partial observability.

1.2 Fault Localization Under Partial Observability

Learning-based methods remove the need for an exact circuit model. Livani et al. (2013), for example, identify the faulted line of the IEEE 123-bus feeder with support-vector machines applied to post-fault root mean square (RMS) voltages. Models that take a flat feature vector as input, however, discard the information about which buses are electrically adjacent. This information is restored by graph neural networks (GNNs). They encode the feeder topology as a graph, and message passing shares information between connected buses, so that the model can infer a fault even at a bus that carries no sensor. Chen et al. (2020) apply a spectral graph convolutional network to the IEEE 123- and 37-bus feeders. Other work combines graph encoders with one-dimensional convolution or attention (Nguyen et al., 2022; Ngo et al., 2025), or pairs a variational graph autoencoder with GraphSAGE to recover latent node information under noise (Fan et al., 2024).

Karabulut et al. (2026) added the time axis. Their spatio-temporal graph neural network (STGNN) couples a gated recurrent unit (GRU) that encodes the voltage history of each sensor with a spatial GNN that exchanges information between sensors. The graph is a measured-only graph: its nodes are only the instrumented buses, and its edges follow the feeder topology. The model assigns each short window of measurements to one of 26 classes, namely 25 candidate fault locations and a no-fault class. The authors compare three spatial encoders, a graph convolutional network, GraphSAGE (Hamilton et al., 2017) and GATv2 attention (Brody et al., 2022). This thesis refers to them as RGCN, RGSAGE and RGATv2 and adopts the framework of Karabulut et al. as its starting point.

The benchmark is the IEEE 123-bus feeder, a standard radial test network. Twenty-five of its buses carry a μPMU. Only six of the 25 candidate fault locations coincide with an instrumented bus, so the remaining 19 locations are never observed directly.

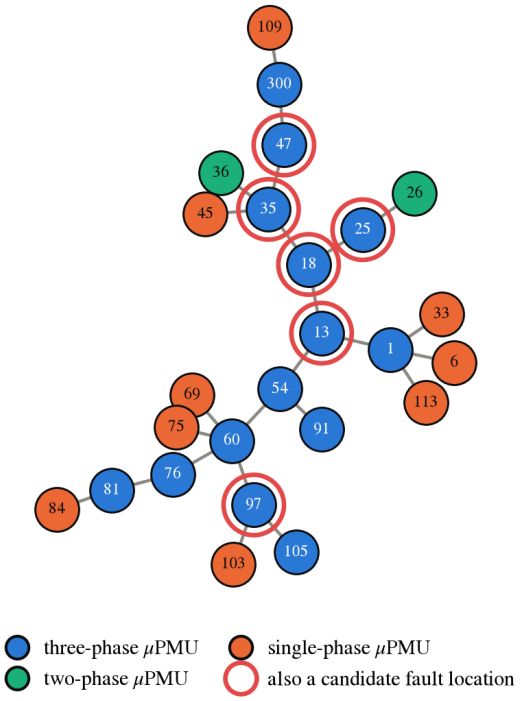


Figure 1.1: The IEEE 123-bus feeder and the measured-only graph used in this thesis.

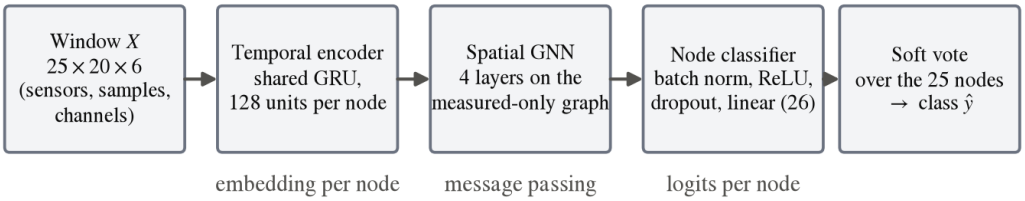


Figure 1.2: Processing template shared by all models: temporal encoder, spatial GNN, node classifier and soft voting.

The model must infer them from the measurements of neighboring sensors. Figure 1.1 shows the feeder and the measured-only graph, and Figure 1.2 shows the processing template that all models in this thesis share.

1.3 Problem Definition

1.3.1 Fault Localization Task

Let $N = 25$ be the number of instrumented buses. A window of measurements contains $S = 20$ consecutive samples taken 1 ms apart. Each sample of each sensor has $F = 6$

channels: the RMS voltage magnitudes of the three phases, $V_{v,s,1}$, $V_{v,s,2}$ and $V_{v,s,3}$, and three availability-mask channels $m_{v,1}$, $m_{v,2}$ and $m_{v,3}$. A mask channel equals one when the corresponding phase is measured and zero when the phase is not installed or the sensor is lost. One window is therefore a tensor $X \in \mathbb{R}^{N \times S \times F}$. The measured-only graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ has the N sensor buses as nodes and 24 edges that form a tree.

The task is to learn a model

$$f_\theta : \mathbb{R}^{N \times S \times F} \to \{0, 1, \ldots, 25\}, \qquad \hat{y} = f_\theta(X), \tag{1.1}$$

where class 0 means that no fault is present and classes 1 to 25 denote the faulted bus. The model returns one prediction per window. The primary performance measure is the macro-averaged F1 score over the 26 classes, called macro-F1 in the remainder of the thesis. Chapter 3 defines it precisely.

1.3.2 Sensor Loss at Inference

A deployed μPMU can stop reporting because of a communication failure or a hardware fault. The thesis models this event by selecting a set $\mathcal{D}$ of k sensors and setting all channels of the selected sensors to zero. The model parameters stay fixed, so the evaluation measures how the model behaves when the input differs from the input seen in training. The thesis varies $k \in \{0, 1, 2, 5, 10\}$, which removes between 0 and 40% of the sensors. The central question is how much the macro-F1 score falls as k grows and how the loss can be reduced.

1.3.3 Measurement Noise and Unseen Fault Conditions

Real measurements carry noise. The thesis adds Gaussian noise to the voltage channels of the sensors that still report. The noise level is given in per-unit voltage, so that it has the same physical meaning for every phase. Together with the sensor loss of the

previous subsection, the two perturbations of the input are

$$\tilde{X}_{v,s,c} = \begin{cases} 0, & v \in \mathcal{D}, \\ X_{v,s,c}, & v \notin \mathcal{D}, \end{cases} \qquad |\mathcal{D}| = k, \tag{1.2}$$

$$\hat{V}_{v,s,p} = V_{v,s,p} + \varepsilon_{v,s,p}, \qquad \varepsilon_{v,s,p} \sim \mathcal{N}(0, \sigma^2), \quad v \notin \mathcal{D},\ m_{v,p} = 1. \tag{1.3}$$

The training data cover only three fault resistances. A model that has learned from data alone has no knowledge of the electrical laws that connect a fault to the voltages at the sensors, so it has no principled way to handle conditions that the data do not cover. This observation motivates the second part of the thesis, which asks whether a fixed physical model of the feeder can supply information that the learned model lacks.

1.4 Conditions Beyond the Training Distribution

A monitoring system leaves the controlled conditions of a laboratory benchmark as soon as it is deployed. Three conditions are of particular interest in this thesis.

- **Sensor loss.** A μPMU depends on a communication link and on periodic maintenance. On a feeder that carries 25 units, losing a few of them is an ordinary operating event, not a rare one.
- **Measurement noise.** Field measurements are never exact. The accuracy class of a synchrophasor measurement device limits the total vector error, and the synchrophasor standard IEEE C37.118 sets this limit at one percent.
- **Unseen fault conditions.** A fault resistance in the field can take any value, whereas a training corpus contains a finite set of values.

The measurements reported in Chapter 4 show that a purely data-driven STGNN is sensitive to the first two conditions. The best baseline, RGSAGE-Max, reaches a macro-F1 of 0.846 when all sensors report. When 10 of the 25 sensors are removed, its macro-F1 falls to 0.048, so it keeps only 5.7% of its clean score. When the single sensor nearest to the fault is removed, the score falls to 0.448, compared with 0.736 when one randomly chosen sensor is removed. Noise with a standard deviation of 0.01 p.u., which equals the one percent limit of the standard, lowers the score from 0.846 to 0.345. Figure 1.3 summarizes the three conditions.

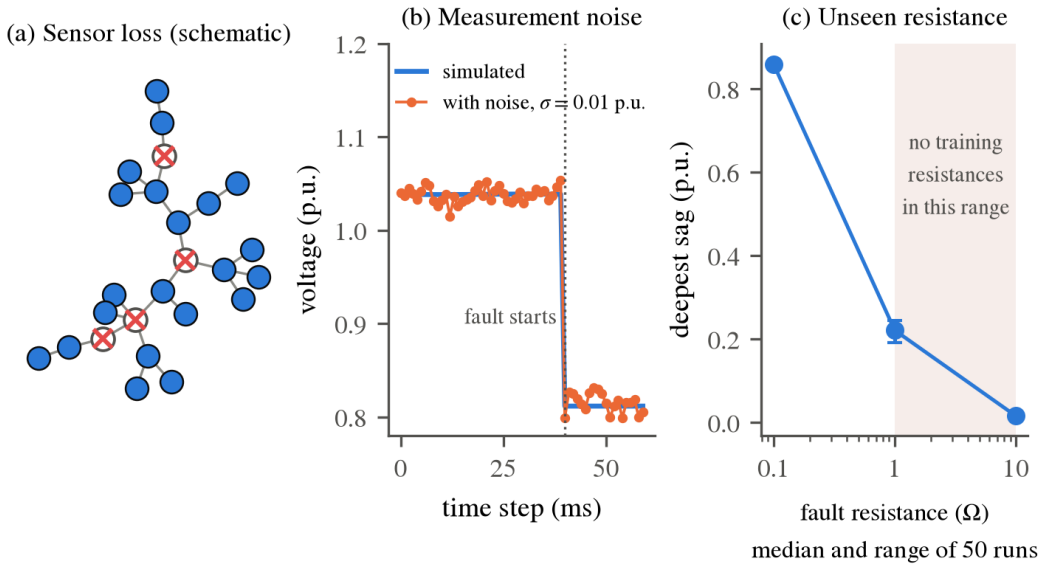


Figure 1.3: Conditions that a deployed monitoring system meets but a clean benchmark does not. Panel (a) is a schematic. Panels (b) and (c) use the simulated corpus: a three-phase fault at bus 101, and noise added with the method of Section 3.6.

Table 1.1: Positioning of the closest prior work. A check mark means that the item is covered, a dash means that it is not studied, and a tilde means that it is covered in part.

Work	Approach	Several models	Sparse sensors	Node loss at test	Remedy
Chen et al. (2020)	Spectral graph convolution, static	–	∼	∼	✓
Zhang et al. (2025)	Graph imputation with contrastive learning	–	✓	∼	✓
Karabulut et al. (2026)	STGNN with RGCN, RGSAGE and RGATv2	✓	✓	–	–
This thesis	STGNN with node-loss study and physics operator	✓	✓	✓	✓

1.5 Research Gap

Table 1.1 positions the closest prior work along the axes that matter here: comparison of several architectures, sparse observability, evaluation with sensors that fail after deployment, and a remedy that reduces the resulting loss.

The table shows three gaps.

1. **No study of sensor loss at inference for STGNN localization.** Karabulut et al. (2026) evaluate their models with a complete and fixed sensor set. Chen et al. (2020) harden a static, single-snapshot classifier against measurement noise and data loss, and Zhang et al. (2025) reconstruct missing measurements with graph imputation, but neither studies a spatio-temporal model whose sensors fail after deployment. To the best of our knowledge, no earlier study measures how STGNN-based localization degrades when whole sensors fail after deployment.

2. **No comparison of mitigation strategies under inference-time failure.** The imputation method of Zhang et al. (2025) assumes that the set of unmeasured buses is a fixed property of the network and is known when the imputer and the classifier are built. An inference-time failure does not provide this knowledge. It is therefore open whether a training-time method, an architectural change or an imputation step recovers the lost accuracy best.

3. **No test of fixed physical knowledge in a learned localizer under noise and loss.** Physics-based methods need dense measurements, and learned methods ignore the electrical laws. Whether a fixed physical model of the feeder complements a learned model under noise, sensor loss and unseen conditions has not been measured on this task. The thesis includes its own screening of three ways to add first-order physical knowledge through the graph, a prior and the loss. None of them improved on the learned model, which shows that the form in which physics enters matters.

1.6 Research Questions and Objectives

The thesis addresses five research questions.

RQ1. How much does the accuracy of STGNN localization fall when μPMUs are lost at inference, and does clean accuracy predict robustness?

RQ2. Can a change in training, an architectural gate or an imputation step recover the lost accuracy, and at what cost?

RQ3. Do the conclusions hold under harder loss patterns, under measurement noise and on unseen fault scenarios?

RQ4. How much information about the fault location do 25 magnitude-only sensors carry when it is measured with a physical model of the feeder, and does this information complement a learned model?

RQ5. Can the physical model be fused with the learned model without adding capacity or leaking labels, and under which perturbations does the fusion help?

The objectives that follow from these questions are:

O1. Generate a corpus of the IEEE 123-bus feeder and benchmark five STGNN-family models under the methodology of Karabulut et al., with several training seeds.

O2. Quantify the degradation of each model under inference-time sensor loss and relate it to the clean accuracy.

O3. Develop dropout-aware training and compare it with neighbor-mean imputation, a learned imputer and an availability-gated architecture.

O4. Stress-test the conclusions with worst-case, persistent and clustered sensor loss, with measurement noise, with a wider training grid, with a classical non-graph baseline and with a split that holds out whole fault scenarios.

O5. Build a fixed physical operator for the feeder, measure the identifiability of the fault location, and evaluate physics-only localization, physics-based imputation and inference-time fusion.

O6. Design a physics-consistent model that adds a single learned scalar to the dropout-aware backbone, verify its design, and compare it with a noise-augmented reference.

1.7 Contributions

The thesis makes the following contributions. The results on dropout-aware training have been accepted for publication (Sangameswar and Anbazhagan, 2026).

1. **A multi-seed benchmark.** Five models of the STGNN family are trained on a regenerated corpus of 1.65 million windows at three seeds each. The thesis reports its own numbers and does not claim to reproduce those of the base paper.

2. **A measurement of node-loss degradation.** Degradation curves for all five architectures show that clean accuracy is an unreliable indicator of robustness: the most accurate model degrades fastest. Retention and the area under the degradation curve make the comparison quantitative.

3. **Dropout-aware training.** Randomly masking sensors during training raises the retention of RGSAGE-Max at 40% sensor loss from 5.7% to 80.3%. The method leaves the number of parameters and the inference latency unchanged, improves the robustness of all five architectures, and outperforms neighbor-mean imputation and a learned imputer. The availability-gated sensor-dropout-robust spatio-temporal graph neural network (SDR-STGNN) matches the method but does not exceed it. The thesis reports this negative result as it is.

4. **A broader robustness analysis.** The conclusions are tested under worst-case, persistent and clustered sensor loss, under measurement noise, with a wider training grid and on a scenario-held-out split. Dropout-aware training does not help against measurement noise, which identifies the limit of a mask-based remedy.

5. **A physical operator and its use.** A table of fault signatures solved with a circuit simulator defines a fixed, parameter-free residual that depends only on the measurements. The thesis uses it to measure identifiability, as a training-free localizer called physics signature matching (PSM), for imputation of lost sensors and for inference-time fusion with the learned model.

6. **A physics-consistent model.** The physics-consistent spatio-temporal graph neural network (PC-STGNN) adds one learned scalar to the dropout-aware backbone. Tests show that it reduces exactly to the backbone when the physical terms are switched off and that the gradient reaches the network weights and not the fixed table.

7. **A careful evaluation protocol.** The thesis uses identical splits and mask seeds for all models, paired tests over training seeds with multiple-comparison correction, a bootstrap over events instead of windows, and reference bounds that include a trivial classifier and an upper limit set by windows that contain no fault signal.

1.8 Scope and Assumptions

The conclusions of the thesis hold within the following limits.

- All data come from simulation of one feeder, the IEEE 123-bus feeder in its default topology. The thesis does not claim that the results transfer to other feeders or to field data.

- The measurements are RMS voltage magnitudes obtained from a quasi-static solver, so they contain no electromagnetic transients, no traveling waves, no direct-current offset and no sub-cycle content. Real-time localization on field streams is not demonstrated.

- The placement of the 25 sensors is taken unchanged from the base paper. The thesis does not optimize the placement.

- The corpus is noise-free. Noise appears only as a controlled perturbation that is added to the measurements.

- The corpus uses a single global load multiplier for each simulated event and three fault resistances. The 70/15/15 partition of the corpus is made over events, so each fault scenario appears in all three splits with different load levels. A separate split that holds out whole scenarios tests the generalization to unseen scenarios.

- The signature table is built with the same simulator that generated the corpus. A comparison on clean data therefore favors the physical operator, and the thesis draws its conclusions from the perturbed conditions.

- The physical operator is called physics-consistent and not physics-informed in the sense of a differential-equation residual. The thesis makes no claim about Kirchhoff residuals or power-flow constraints.

- Every result with several seeds uses three training seeds. This number limits the statistical power, and the thesis states which orderings are consistent observations and which are significant.

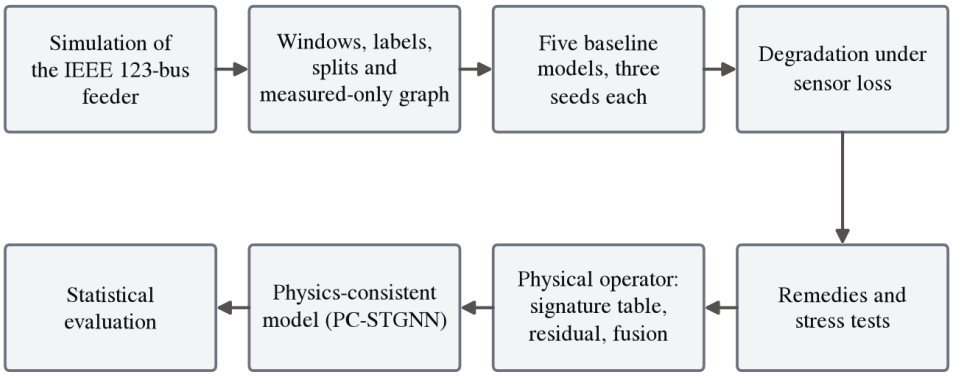


Figure 1.4: Overview of the research workflow of the thesis.

1.9 Methodology Overview

Figure 1.4 gives an overview of the research workflow. The work starts with the simulation of the feeder and the construction of windows, labels, splits and the measured-only graph. Five baseline models are then trained and evaluated under increasing sensor loss. The mitigation methods and the additional stress tests build on the same corpus and the same splits. In the second part, a signature table of the feeder defines the physical operator, which is analyzed on its own before it is combined with the learned model.

Four rules govern all experiments. First, only one factor changes between two compared models. Second, all models use the same frozen data, the same splits and the same sensor-loss masks. Third, hyperparameters and checkpoints are selected on the validation split only, and the test split serves only for reporting. Fourth, inexpensive checks that need no training are run before an expensive training run, and every new component is checked against the model without it before it is trusted.

1.10 Organization of the Thesis

The remaining chapters are organized as follows. Chapter 2 reviews the literature on impedance-based, traveling-wave and learning-based fault localization, on graph neural networks for power systems, on missing and noisy measurements, and on physics-aware learning, and it derives the research gap in more detail. Chapter 3 describes the proposed work: the simulated corpus, the five baseline architectures, the sensor-loss

evaluation protocol, dropout-aware training, the availability-gated model, the physical operator and the physics-consistent model, together with the evaluation metrics and the statistical procedure. Chapter 4 presents the results and their interpretation: the baseline benchmark, the degradation curves, the comparison of mitigation methods, the stress tests, the analysis of the physical operator and the evaluation of the physics-consistent model. Chapter 5 summarizes the findings, states the limitations and outlines future work.

Chapter 2

LITERATURE SURVEY

This chapter reviews the work on which the thesis builds. It follows the development of fault localization from methods that rely on circuit physics, through learning methods that ignore the network structure, to graph-based and spatio-temporal models. It then examines how the literature treats missing and noisy measurements and how physical knowledge has been combined with graph learning. The chapter ends with a summary of what the literature leaves open.

2.1 Overview of Approaches

The approaches to fault localization in distribution grids fall into four families. Circuit-based methods compute the position of the fault from impedances or from wave propagation. Learning methods with hand-crafted features map signatures of the measurements to a fault location. Graph-based learning methods represent the feeder as a graph, and spatio-temporal variants add a temporal encoder. Physics-aware learning methods combine a learned model with knowledge of the electrical laws. Each family relaxes an assumption of the one before it. Learning methods remove the need for an exact circuit model, graph models restore the topology that feature vectors discard, and spatio-temporal models add the evolution of the signal over time. Figure 2.1 summarizes the families, and Table 2.1 lists a representative work and the main limitation of each family for the task of this thesis.

2.2 Circuit-Based Fault Location

Impedance-based methods are the longest established family. They measure voltage and current at one point of the feeder, usually the substation, compute the apparent impedance and convert it into a distance. A well-known difficulty is the multiple-estimation problem: on a branching feeder, a single measured impedance corresponds

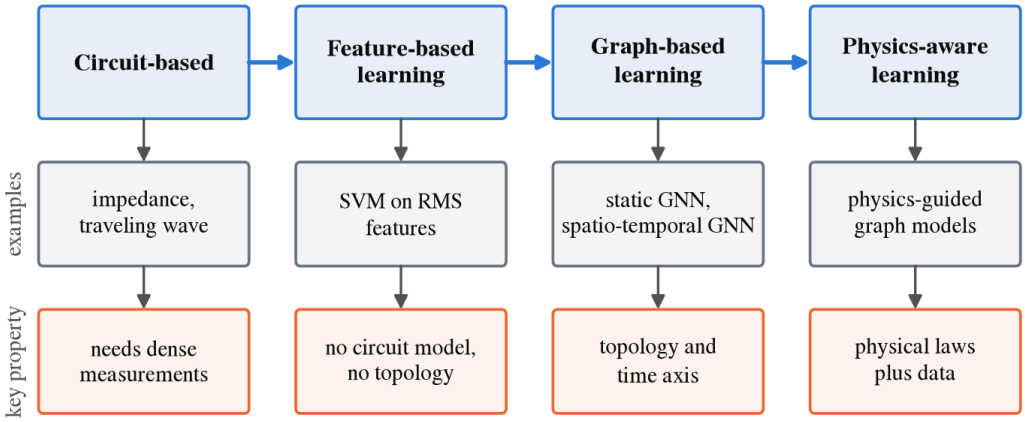


Figure 2.1: Families of fault localization methods reviewed in this chapter.

Table 2.1: Families of fault localization methods, a representative work, and the main limitation for fault localization under partial observability.

Family	Representative work	Main limitation for this task
Impedance-based	Krishnathevar and Ngu (2012)	Ambiguity on branching feeders; needs accurate line parameters; affected by fault resistance
Traveling-wave	Liu et al. (2023)	Needs fast sampling and dense measurement coverage
Feature-based learning	Livani et al. (2013)	Flat feature vector; the adjacency of buses is not used
Static graph learning	Chen et al. (2020), Fan et al. (2024)	Single snapshot; no time axis
Spatio-temporal graph learning	Karabulut et al. (2026)	Evaluated with a complete, fixed set of sensors
Imputation-based learning	Zhang et al. (2025)	Assumes a known, fixed set of unmeasured buses
Physics-aware graph learning	Li and Deka (2021)	Observability is sparse but fixed; noise and sensor loss after deployment are not studied

to several candidate positions. Krishnathevar and Ngu (2012) give a generalized one-end formulation that resolves this ambiguity. The accuracy of such methods depends on accurate line parameters, and the fault resistance disturbs the estimate.

Traveling-wave methods follow a different principle. A fault launches a wavefront that travels along the line, and the arrival times at the measurement points identify the fault position. Liu et al. (2023) locate faults on multi-branch distribution networks by sharpening the arrival instants of the wavefront with an improved ensemble empirical mode decomposition and the Teager energy operator. These methods are largely inde-

pendent of the fault resistance, but they need devices that sample at a very high rate and measure at many points.

Both families have two properties in common. They assume a measurement coverage that is dense enough to observe the impedances or the wavefronts of interest, and neither represents which buses are electrically adjacent. Neither assumption holds on a feeder that carries a few μPMUs, which is the setting of this thesis.

2.3 Feature-Based Learning Methods

Learning methods avoid the need for an exact circuit model. They learn the mapping from the measured signature of a fault to its location from simulated or recorded events. Livani et al. (2013) identify the faulted line of the IEEE 123-bus feeder with a support-vector machine (SVM) that uses post-fault RMS voltages and fault-current-indicator signals. The model handles the nonlinear relation between signature and location without a circuit model, but it takes a flat feature vector. It cannot use the knowledge that a fault between two buses depresses the voltage at their neighbors in a structured way. This limitation motivates the graph-based methods of the next section.

2.4 Graph Neural Networks for Fault Location

2.4.1 Message Passing on the Feeder Graph

A GNN computes a representation for each node of a graph by repeated exchange of information with the neighbors of the node. A general layer has the form

$$h_v^{(l+1)} = \phi^{(l)}\Big(h_v^{(l)},\ \mathrm{AGG}\big(\{h_u^{(l)} : u \in \mathcal{N}(v)\}\big)\Big), \tag{2.1}$$

where $h_v^{(l)}$ is the representation of node v after l layers, $\mathcal{N}(v)$ is the set of neighbors of v, AGG is a permutation-invariant aggregation, and $\phi^{(l)}$ is a learnable update. The choice of AGG defines the architecture. A graph convolutional network (GCN) (Kipf and Welling, 2016) aggregates with a normalized sum, GraphSAGE (Hamilton et al., 2017) concatenates the node with a mean or max of its neighbors, and GATv2 (Brody et al., 2022) weights the neighbors with learned attention coefficients. After L layers,

a node holds information from nodes at most L hops away. This property is what lets a GNN infer a fault at a bus that carries no sensor, because the evidence from the nearest sensors reaches the node through the graph. Liao et al. (2021) survey the use of GNNs in power systems and list fault diagnosis, time-series prediction and power flow calculation among the main applications.

2.4.2 Graph-Based Fault Location Models

Chen et al. (2020) apply a spectral graph convolutional network with Chebyshev filters to the IEEE 123- and 37-bus feeders. They add a data augmentation against measurement noise and data loss, which is the closest precedent for the training-time mitigation studied in this thesis. Their model, however, classifies a single snapshot over the graph of the full grid. Fan et al. (2024) pair a variational graph autoencoder with GraphSAGE to recover latent node information under noise, and they also classify a single snapshot.

A temporal encoder supplies the time axis. Nguyen et al. (2022) combine a one-dimensional convolutional network with a graph convolutional network, and Ngo et al. (2025) combine one with graph attention, both for fault detection and identification in distribution systems. In both models the convolution acts on the time axis of the measurements and the graph network on the spatial axis.

Li and Deka (2021) propose a physics-preserved graph network for fault location with limited observation and limited labels. A first stage learns an embedding of the whole network from a few measured nodes. A second stage relates labeled and unlabeled samples to improve the location accuracy. They evaluate it on the IEEE 123- and 37-node feeders with varying loads and topology. In this model the observability is sparse but fixed, and the study does not vary the set of available sensors after training.

2.5 Spatio-Temporal Graph Models and the Measured-Only Graph

Karabulut et al. (2026) give the most complete treatment of spatio-temporal learning for fault location under partial observability, and their framework is the starting point of this thesis. Their pipeline combines a recurrent network that encodes the voltage

history of each sensor with a spatial GNN. They benchmark GraphSAGE and GATv2 encoders, called RGSAGE and RGATv2, against a graph convolutional encoder, called RGCN, and against a recurrent baseline, on the IEEE 123-bus feeder. The letter R in these names refers to the recurrent front end. A second contribution concerns the graph itself. Earlier work built the graph from the full topology of the grid. Karabulut et al. propose a measured-only graph, whose nodes are only the instrumented buses. They report that all spatio-temporal variants outperform the purely recurrent baseline by up to 11 percentage points of F1, that the variants are more stable across runs, and that the measured-only graph reduces the training time six-fold and improves the F1 score by up to 11 points compared with the full topology. Figure 2.2 contrasts the two graph definitions.

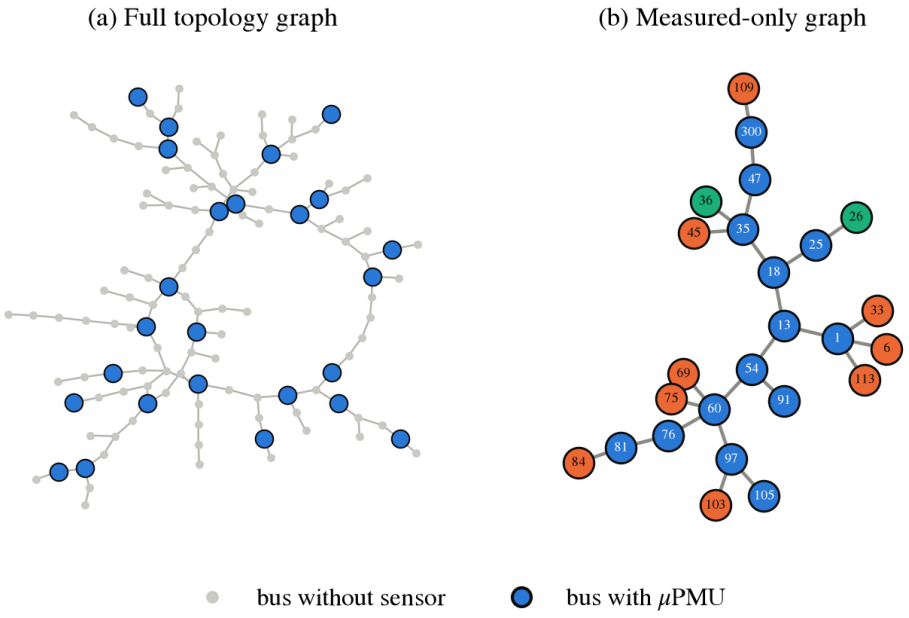


Figure 2.2: Full-topology graph and measured-only graph.

A companion study by the same authors (Karabulut et al., 2025) benchmarks the same family of encoders for fault detection and examines how well they generalize to grid configurations that differ from those used in training. They report that RGATv2 loses about 12% of its F1 score across topology settings, other variants of the family lose up to about 25%, and purely recurrent models lose up to about 60%. This study varies the topology. It does not vary the availability of the sensors.

The same template of a temporal encoder followed by a graph network appears in other domains. Sachin et al. (2025) cast spatio-temporal forecasting as an adap-

tive encoder-decoder graph network. Jayapal and Annamalai (2025) build a graph over segmented lymph-node regions to classify cancer metastasis, and Asiri and Somasundaram (2025) identify illicit Bitcoin transactions with a graph convolutional network on the transaction graph. Both report gains over models that do not use the graph. These examples show that the design questions raised in this thesis are not specific to power systems.

2.6 Missing and Noisy Measurements

2.6.1 Imputation and Reconstruction

The most developed treatment of missing data in fault location is the work of Zhang et al. (2025). A training-free graph Dirichlet imputation reconstructs the missing bus measurements by minimizing the Dirichlet energy of the graph, and a graph convolutional network trained with contrastive learning keeps the fault signatures separable under the resulting scarcity. The method assumes that the set of unmeasured buses is a fixed property of the network that is known when the imputer and the classifier are built. A sensor that fails after deployment does not satisfy this assumption.

The idea behind this imputation is not specific to fault location. Rossi et al. (2021) propose Feature Propagation, which minimizes the Dirichlet energy to fill missing node features, and they report that the accuracy of node classification falls by only about 4% relative when 99% of the features are missing. The setting is node classification on benchmark graphs, where the missing features are fixed before training. The relation to the loss of a whole sensor at inference time is therefore indirect.

2.6.2 Robustness to Sensor Failure

Work on sensors that fail at inference time is rare outside power systems. Kirthan et al. (2026) combine diffusion graph convolutions and dilated temporal convolutions with Monte Carlo dropout to separate aleatoric from epistemic uncertainty in traffic flow prediction, and they report that the model holds up better than three spatio-temporal baselines when sensors drop out. The task is a regression with dense sensing, and a lost detector mainly degrades the prediction at its own location. In fault localization,

the predicted quantity is a discrete location that is usually not instrumented, so a lost μPMU withdraws evidence about buses other than its own.

2.6.3 Masking During Training

Randomly removing parts of a graph during training is a known regularizer. DropEdge (Rong et al., 2020) removes edges at random to slow down over-smoothing and over-fitting in deep graph convolutional networks. Masking measurement nodes during training is a related operation on the input features instead of the edges. The thesis does not claim the masking primitive as new. Its contribution is to frame sensor loss as a mismatch between training and deployment conditions and to measure the effect of the remedy against several alternatives.

2.7 Physics-Aware Learning for Power Systems

Physics-informed graph learning incorporates physical rules into graph learning, and Peng et al. (2022) review the methods in a unified framework. Nyangon (2026) review hybrid physics-informed architectures for electricity systems, including physics-informed neural networks (PINNs) and their graph-based variants, and they name training instability, computational cost and the absence of standard benchmarks as persistent challenges. The mechanisms in this literature differ widely. A physical term can enter the loss, the architecture, the graph construction or the features. Okoyomon et al. (2025) compare three such strategies for voltage prediction in distribution grids, namely power-flow-constrained losses, complex-valued networks and residual-based task reformulation, and they assess both standard and out-of-distribution performance. Dogoulis et al. (2025) impose Kirchhoff's current law (Kirchhoff's current law (KCL)) as a hard constraint on the output of a graph network by projection onto hyperplanes, which yields power flow predictions without violations of the law.

Two observations apply to the present task. First, most of these methods need quantities that the data of this thesis do not contain. A Kirchhoff residual needs complex voltages or currents, whereas the μPMUs of the benchmark report RMS voltage magnitudes only. Residuals of this kind cannot be formed from magnitudes alone. Second, the approach of Li and Deka (2021) shows that physics can serve as a guide for the

graph construction under sparse observation, but it does not examine sensors that fail after deployment or noise in the measurements.

The thesis therefore uses a different form of physical knowledge. A circuit simulator solves the feeder for every candidate fault, and the resulting signatures define a fixed residual between a measurement and each hypothesis. This residual needs no complex quantities, and it contains no learned parameter. The thesis calls this knowledge physics-consistent and does not call it physics-informed in the sense of a differential-equation residual.

2.8 Summary and Research Gap

Table 2.1 in Section 2.1 lists the families and their limitations.

The survey leads to four observations that define the research gap of this thesis.

1. Spatio-temporal graph models are the most capable family for fault localization under partial observability, but the existing evaluations assume that every sensor reports at all times. No study measures how these models degrade when whole sensors fail after deployment.

2. The remedies in the literature for missing data assume a fixed pattern of missingness that is known in advance, or they address noise and data loss in static classifiers. It is not known whether a training-time method, an architectural change or an imputation step recovers the accuracy lost to an inference-time sensor failure, and at what cost.

3. The evaluations use a single missingness model or none. Whether a remedy transfers to worst-case, persistent, spatially clustered or noisy conditions is not measured.

4. Physics-aware methods rely on complex quantities or on fixed observability. How much a fixed physical model of the feeder adds to a learned localizer that sees only magnitudes, and under which perturbations it adds it, has not been measured on this task.

The next chapter describes the methods that the thesis uses to close these gaps.

Chapter 3

PROPOSED WORK

This chapter describes the methods of the thesis in the order in which they build on one another. Section 3.1 describes the simulated corpus. Section 3.2 defines the five baseline models, and Section 3.3 defines the protocol that measures their behavior under sensor loss. Sections 3.4 to 3.6 describe dropout-aware training, the availability-gated model and the comparison and stress-test methods. Sections 3.7 and 3.8 describe the physical operator and the physics-consistent model. Section 3.9 describes the checks of the evaluation pipeline.

3.1 Simulation and Dataset

3.1.1 Feeder and Scenario Design

The corpus is generated by simulation of the IEEE 123-bus distribution feeder, a standard radial benchmark with a nominal line-to-line voltage of 4.16 kV at 60 Hz. The simulator is OpenDSS, driven through its Python interface. The simulation runs in daily (quasi-static time-series) mode with a step of 1 ms. Each recorded sample is therefore a converged power-flow solution and not a single operating-point snapshot. The corpus describes the post-inception RMS depression of the voltage and its spatial pattern over the feeder. It does not contain the electromagnetic transient of the fault, so the accuracies reported in this thesis apply to this class of signal.

The scenarios follow the factorial design of Karabulut et al. (2026). Faults are injected at 25 candidate buses, with 11 fault types and 3 fault resistances, which gives 825 scenarios. The fault types are the three line-to-ground faults, the three line-to-line faults, the three double-line-to-ground faults and two three-phase faults, one with and one without ground. The fault resistances are 0.1, 1 and 10 Ω. Each scenario is simulated 50 times. A run draws one global load multiplier η from the uniform distribution on $[0.5, 1.3]$ and applies it to every load of the feeder, so the 50 runs of a scenario differ only in the load level. The seed of each run is a deterministic function of the bus, the

fault type, the resistance and the run index, so every event can be regenerated exactly. Table 3.1 summarizes the configuration.

Table 3.1: Configuration of the simulated corpus.

Item	Value
Feeder and simulator	IEEE 123-bus, 4.16 kV, 60 Hz; OpenDSS in daily mode with a 1 ms step
Candidate fault buses	25
Fault types	11 (3 line-to-ground, 3 line-to-line, 3 double-line-to-ground, 2 three-phase)
Fault resistances	0.1, 1 and 10 Ω
Load multiplier	One global factor per run, uniform on $[0.5, 1.3]$
Scenarios and runs	$25 \times 11 \times 3 = 825$ scenarios, 50 runs each, 41,250 events
Record of one event	60 ms: 40 ms before the fault and 20 ms with the fault
Windows	20 samples (20 ms), stride 1 ms, 40 windows per event
Node features	Three RMS phase voltages and three availability-mask channels
Classes	26: 25 fault locations and no fault

3.1.2 Sensors and the Measured-Only Graph

The sensors follow the placement of Karabulut et al. (2026). Twenty-five buses carry a μPMU: buses 1, 6, 13, 18, 25, 26, 33, 35, 36, 45, 47, 54, 60, 69, 75, 76, 81, 84, 91, 97, 103, 105, 109, 113 and 300. Fourteen of them measure all three phases. The other eleven measure one or two phases, so 55 of the 75 phase channels are instrumented. A channel that is not instrumented carries a voltage of zero and an availability mask of zero. Only six of the 25 candidate fault buses (13, 18, 25, 35, 47 and 97) coincide with an instrumented bus. The placement is taken as given and is not optimized.

The measured-only graph has the 25 instrumented buses as nodes. Its edges follow a breadth-first spanning tree of the feeder that is rooted at the substation, bus 150. Each unit is connected to the nearest instrumented unit upstream of it on this tree. One exception keeps the graph connected: a single- or two-phase unit whose nearest upstream unit is also single- or two-phase is not connected to that unit. Two units, at buses 33 and 113, are in this case, and a fallback connects each of them along the shortest feeder path to the unit at bus 1. The result is a connected tree with 24 edges. Figure 1.1 in Chapter 1 shows the graph.

3.1.3 Windows, Labels, Splits and Normalization

An event is recorded for 60 ms. The fault is injected at step 40, so the record has 40 steps before the fault and 20 steps with the fault. Windows of 20 samples are cut with a stride of 1 ms, and the first 40 windows are kept, with start steps from 0 to 39. A window starting at step w covers the steps from w to $w + 19$. Windows 0 to 19 end before the fault and have the label no fault. Windows 20 to 39 end after the fault starts and carry the label of the faulted bus. Window w therefore contains $w - 20$ samples after fault inception. Every event contributes 20 windows of each kind, so exactly half of the 1.65 million windows belong to the no-fault class and the 25 fault classes are equally frequent.

The partition into training, validation and test data is made at the event level, stratified by fault bus, with the proportions 70, 15 and 15 percent. All 40 windows of an event stay in one split, and the measured overlap of events between any two splits is zero. Table 3.2 gives the counts. Each fault scenario appears in all three splits with different load levels, so the split measures generalization to unseen load conditions of a known fault configuration. Section 3.1.5 describes the complementary split that holds out scenarios.

Table 3.2: Event-level partition of the corpus.

Split	Events	Fault windows	No-fault windows	Total windows
Training	28,875	577,500	577,500	1,155,000
Validation	6,200	124,000	124,000	248,000
Test	6,175	123,500	123,500	247,000
Total	41,250	825,000	825,000	1,650,000

The voltage channels are normalized to zero mean and unit variance per phase with statistics of the training split. The mask channels stay binary. The mask channels are static in the clean corpus: they encode which phases each unit measures and are identical in every window. This property matters for the interpretation of dropout-aware training in Chapter 4.

3.1.4 Data Quality and Reference Bounds

Several checks were run on the whole corpus and not on samples. All metadata fields are present for all 41,250 events. After the second post-fault step the voltage level is constant to within 1.1×10^{-7} p.u., so the fault period is quasi-static. The first two post-fault steps show a solver settling of at most 2×10^{-5} p.u. in all events except three ungrounded faults, in which a control device operates and the settling reaches about 7×10^{-3} p.u. These three events affect six of the 1.65 million windows.

Two reference bounds follow from the window design. A classifier that always answers no fault reaches a macro-F1 of 0.0256, which serves as the trivial reference. Window 20 of every fault event carries a fault label but contains no post-fault sample, so it is physically identical to a healthy window. No model can beat chance on these windows, and they lower the best achievable macro-F1 to 0.9744. The windows make up 2.5% of the test split. The accuracy of every trained baseline and mitigation model on this subset lies between 0.0003 and 0.0418 against a chance rate of 0.0385, which shows that no label information leaks into the inputs.

3.1.5 Scenario-Held-Out Split

The event-level split leaves every scenario in all three splits. To test generalization to unseen fault configurations, a second split assigns whole scenarios to one split. The assignment is made within each fault bus, so all 25 fault classes remain present in every split, and what is withheld is the combination of fault type and resistance at that bus. The split has 575 training scenarios, 125 validation scenarios and 125 test scenarios, with no overlap, and it covers 28,750, 6,250 and 6,250 events. The windows and the normalization statistics are those of the original corpus, so the split is the only variable that changes.

3.2 Baseline Architectures

3.2.1 Shared Template

The five baseline models follow the framework of Karabulut et al. (2026). A temporal encoder produces one embedding per instrumented bus, an optional spatial stage ex-

changes information over the measured-only graph, and a shared classifier assigns class probabilities to each node. A soft vote over the nodes yields the prediction for the window. Let $x_v \in \mathbb{R}^{S \times F}$ be the input of node v. A single-layer GRU with 128 hidden units, shared by all nodes, encodes it:

$$h_v^{(0)} = \mathrm{GRU}(x_v) \in \mathbb{R}^{128}, \qquad v = 1, \ldots, N. \tag{3.1}$$

The model that consists of this encoder and the classifier alone is the recurrent baseline, called GRU in the tables. It has no message-passing stage and is therefore not a spatio-temporal graph model.

3.2.2 Spatial Encoders

The four graph models stack $N_{\mathrm{hop}} = 4$ message-passing layers with a hidden dimension of 64 and self-loops. They differ only in the aggregation operator. Let $\tilde{\mathcal{N}}(j) = \mathcal{N}(j) \cup \{j\}$ be the closed neighborhood of node j. RGCN propagates over the symmetrically normalized adjacency $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ (Kipf and Welling, 2016). RGSAGE concatenates the representation of a node with a mean-pooled or max-pooled aggregate of its neighbors (Hamilton et al., 2017), which gives the variants RGSAGE-Mean and RGSAGE-Max. RGATv2 weights the neighbors with learned attention coefficients and uses $H = 4$ heads (Brody et al., 2022). The three updates are

$$h_j^{(l+1)} = \mathrm{ReLU}\Big(\textstyle\sum_{i \in \tilde{\mathcal{N}}(j)} \frac{1}{\sqrt{\tilde{d}_i \tilde{d}_j}}\, h_i^{(l)} W^{(l)} \Big), \tag{3.2}$$

$$h_j^{(l+1)} = \mathrm{ReLU}\Big(W^{(l)} \big[h_j^{(l)} \,\|\, \rho\big(\{h_i^{(l)} : i \in \tilde{\mathcal{N}}(j)\}\big) \big] \Big), \tag{3.3}$$

$$h_j^{(l+1)} = \mathrm{ReLU}\Big(W_{\mathrm{o}}^{(l)} \Big\|_{c=1}^{H} \sum_{i \in \tilde{\mathcal{N}}(j)} \alpha_{ji}^{(l,c)}\, W^{(l,c)} h_i^{(l)} \Big), \tag{3.4}$$

where $\tilde{d}_i$ is the degree of node i including the self-loop, $\|$ denotes concatenation, ρ is the mean or the maximum, and $\alpha_{ji}^{(l,c)}$ is the attention coefficient of head c. The rectifier is omitted after the last layer, whose output passes directly to the classifier. The depth of four layers is a choice of this thesis. A single layer limits the receptive field of a node to its immediate neighbors, and most fault locations lie more than one hop from any instrumented bus. A diagnostic sweep of the depth of RGSAGE-Max confirmed that a

deeper network is needed, as Chapter 4 shows. The depth was selected on RGSAGE-Max alone and then applied to every architecture.

A shared classifier maps the final embedding of each node to 26 logits. It consists of batch normalization, a rectifier, dropout with a rate of 0.35 and a linear layer. At inference the class probabilities of the N nodes are summed, and the class with the largest sum is the prediction for the window:

$$\hat{y} = \arg \max_{c \in \{0,\dots,25\}} \sum_{v=1}^{N} \mathrm{softmax}(z_v)_c. \tag{3.5}$$

Training minimizes the unweighted node-level cross-entropy, in which the label of the window supervises every node,

$$\mathcal{L} = \frac{1}{N} \sum_{v=1}^{N} \mathrm{CE}(z_v, y). \tag{3.6}$$

Class imbalance is handled by the design of the corpus, which is balanced, and not by loss weighting.

3.2.3 Training Configuration

All models share one training configuration, listed in Table 3.3. Batches are formed by an event-chunk sampler that draws windows from a small set of events at a time, so that each compressed event file is read once per epoch, and that mixes healthy and faulted windows in every batch. The checkpoint with the best validation graph-level macro-F1 is kept, and no other early stopping is used. Every model is trained at three seeds, 42, 43 and 44, which control the parameter initialization, the batch order and the masking of dropout-aware training.

3.3 Sensor-Loss Evaluation Protocol

3.3.1 Missingness Model

Robustness to sensor loss is measured entirely at inference time, with all model parameters fixed. This isolates the behavior of a deployed model from any training-time

Table 3.3: Training and software configuration shared by all models.

Item	Setting
Temporal encoder	One GRU layer, 128 hidden units, shared across nodes
Spatial layers	Four message-passing layers, hidden dimension 64
Classifier	Batch normalization, rectifier, dropout 0.35, linear layer with 26 outputs
Optimizer	AdamW, learning rate 10^{-3}, weight decay 10^{-4}
Schedule and clipping	Linear warm-up over the first 3 epochs, constant afterwards; gradient norm clipped at 1.0
Batch size and epochs	32 windows, 30 epochs
Loss and model selection	Unweighted node-level cross-entropy; best validation macro-F1
Seeds	42, 43 and 44
Hardware	One NVIDIA T1000 with 8 GB of memory
Software	Python 3.12, PyTorch 2.5.1 with CUDA 12.1, PyTorch Geometric 2.8, OpenDSSDirect.py 0.9.4, scikit-learn 1.9

remedy. The thesis draws a set $\mathcal{D}$ of k of the 25 sensors for each test window and sets all six input channels of every selected node to zero, as in Eq. (1.2). The mask channels are zeroed together with the voltages because a zero voltage can be a valid reading after normalization, so the mask is the only definitive indicator that a node is unobserved. A unit that fails is thus encoded like a phase that was never installed. A dropped node keeps its edges and takes part in message passing. Its temporal encoder receives an all-zero sequence, and the resulting embedding propagates to its neighbors. This propagation is the phenomenon under study. A policy that removes the edges of dropped nodes exists in the code but is not used, because it would change the topology and the measurement at the same time.

The study uses $k \in \{0, 1, 2, 5, 10\}$, which removes 0, 4, 8, 20 and 40% of the sensors. For $k > 0$ the selected nodes are drawn uniformly at random, independently for every window. The pseudo-random seed of a batch is derived from the level k, the index of the mask realization and the index of the batch, so that every architecture sees an identical sequence of dropped sensors on the same frozen test split. Each point of a degradation curve is averaged over three mask realizations. The three realizations are averaged within each training seed before any statistical test, so that mask realizations are not treated as independent samples.

3.3.2 Metrics

The primary metric is the macro-averaged F1 score over the 26 classes. Each test window receives one prediction by the soft vote of Eq. (3.5). The predictions of all 247,000 test windows are pooled, without aggregation over the 40 windows of an event. The per-class F1 scores are averaged without weighting:

$$\mathrm{F1}_{\mathrm{macro}} = \frac{1}{26} \sum_{c=0}^{25} \frac{2\,\mathrm{Pr}_c\,\mathrm{Re}_c}{\mathrm{Pr}_c + \mathrm{Re}_c}, \tag{3.7}$$

where Pr_c and Re_c are the precision and the recall of class c. The no-fault class holds half of the windows but only one twenty-sixth of the weight, and the macro average is used so that the 25 fault classes dominate the score. The node-level macro-F1 reported for the baselines treats the prediction of each node as a separate sample.

Two quantities describe the shape of a degradation curve. The retention at level k is the score relative to the clean score, and the normalized area under the retention curve (AUDC) is its normalized integral over the sweep:

$$\mathrm{Ret}(k) = \frac{\mathrm{F1}(k)}{\mathrm{F1}(0)}, \tag{3.8}$$

$$\mathrm{AUDC} = \frac{1}{k_{\max} - k_{\min}} \int_{k_{\min}}^{k_{\max}} \mathrm{Ret}(k)\,\mathrm{d}k \in [0, 1], \tag{3.9}$$

with $k_{\min} = 0$ and $k_{\max} = 10$ and trapezoidal integration over the five levels. A larger value means slower degradation. Both measures are relative to the clean score of the model itself. They compare how gracefully a model degrades and not how accurate it is, and a weak model can obtain a flattering value. The thesis therefore also reports the absolute area, which is the same integral applied to the raw macro-F1, and it reads every score against the trivial reference of 0.0256 and the ceiling of 0.9744.

3.3.3 Statistical Procedure

Every comparison between variants is a mean over three training seeds, which is a small sample, so the thesis states which differences survive a test. The training seed is the paired unit, because all variants are trained from the same three seeds on the same data. The test is a paired two-sided t-test on the seed-wise differences with two degrees

of freedom, reported with a 95% confidence interval. Tests that answer one question form a family, and the Holm-Bonferroni correction is applied within each family. Every reported p-value is the adjusted value p_{Holm}. A Wilcoxon signed-rank test is not used, because its smallest attainable two-sided p-value at three seeds is 0.25. A complementary cluster bootstrap resamples the 6,175 test events and not the windows, because the 40 windows of an event are correlated. The bootstrap shows whether the limit of an ordering is the number of training seeds or the size of the test set.

3.4 Dropout-Aware Training

The baselines are trained with all 25 sensors available but are evaluated with some sensors missing, so they are tested on inputs that differ from the inputs of training. Dropout-aware training removes this mismatch by applying the evaluation missingness model during training. For every mini-batch, a dropout level k is drawn uniformly from $\mathcal{K} = \{0, 1, 2, 5\}$. Then, independently for every window of the batch, k nodes are drawn uniformly without replacement and all six channels of the selected nodes are set to zero. The masking generator is reseeded at the start of every epoch. Because k varies from batch to batch, every epoch mixes complete and incomplete observations, so the network cannot depend on the availability of any individual sensor. Figure 3.1 shows the setup and Algorithm 1 states the procedure.

Algorithm 1 Dropout-aware training. Only the masking in steps 4 and 5 differs from standard training.

Require: training batches $\mathcal{B}$, dropout levels $\mathcal{K}$, number of epochs E, model f_θ

1: **for** $e = 1$ **to** E **do**
2: reseed the masking generator
3: **for** each mini-batch $(X, y) \in \mathcal{B}$ **do**
4: draw k uniformly from $\mathcal{K}$
5: for every window of the batch, draw k nodes uniformly and zero all six channels of these nodes, which gives $\tilde{X}$
6: compute the node logits $Z = f_\theta(\tilde{X})$ and the loss of Eq. (3.6)
7: clip the gradient norm at 1.0 and take an AdamW step
8: **end for**
9: **end for**

The augmentation touches only the input features. It changes neither the graph, the architecture, the loss nor the optimizer, so it applies without modification to every

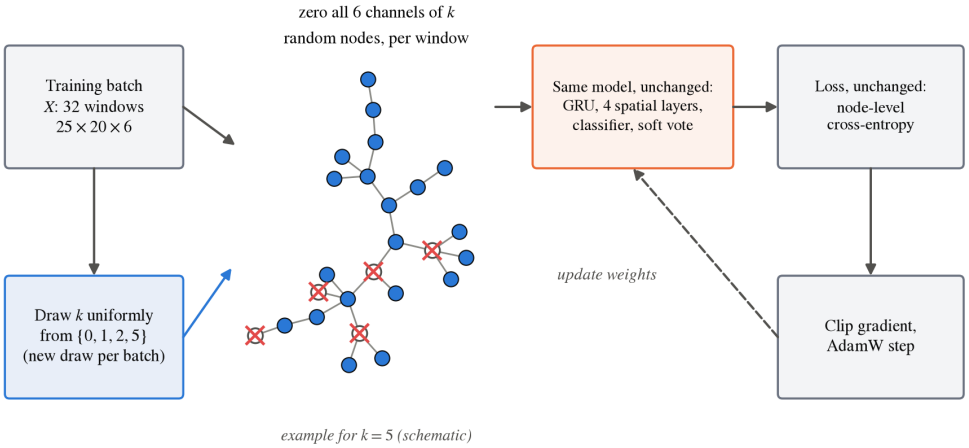


Figure 3.1: Dropout-aware training. Only the masking of the input differs from standard training. The example graph is a schematic.

baseline and to the availability-gated model of the next section. The masking operation is related to DropEdge (Rong et al., 2020) and is not claimed as new; the contribution is to frame sensor loss as a train-test mismatch and to measure the remedy against alternatives. The main configuration trains with $\mathcal{K} = \{0, 1, 2, 5\}$ and therefore never shows the level $k = 10$ of the evaluation. A variant with $\mathcal{K} = \{0, 1, 2, 5, 10\}$, called DA-wide, tests whether including the largest level helps.

3.5 SDR-STGNN: Availability-Gated Aggregation

The sensor-dropout-robust spatio-temporal graph neural network, SDR-STGNN, addresses sensor loss through the architecture and serves as an ablation against dropout-aware training. All five baselines receive the mask as input, yet none of the aggregation operators of Eqs. (3.2) to (3.4) uses it, so a dropped node keeps contributing its zero-valued embedding as a non-informative message. SDR-STGNN builds on RGSAGE-Max and makes the aggregation depend on availability. For every window, a node is available if any of its mask channels is active at any step:

$$a_i = \mathbf{1}\Big\{ \max_{s} \max_{p \in \{1,2,3\}} m_{i,p}(s) > 0 \Big\} \in \{0, 1\}. \tag{3.10}$$

The max-pooled aggregation uses only available source nodes, and the same indicator gates the self-representation:

$$h'_j = \mathrm{ReLU}\Big(W\big[a_j h_j \,\|\, \mathrm{agg}_j\big]\Big), \qquad \mathrm{agg}_j = \max_{\substack{i \in \tilde{\mathcal{N}}(j) \\ a_i = 1}} h_i, \tag{3.11}$$

where $\mathrm{agg}_j = 0$ if no node of the closed neighborhood of j is available. Dropped nodes send no messages. The availability indicators are computed once from the input and reused in all four layers. The gate adds no parameters. When every sensor is available ($a_i = 1$ for all i), Eq. (3.11) reduces exactly to the RGSAGE-Max update, so the two models are identical at $k = 0$. SDR-STGNN is trained with the dropout-aware augmentation, so that the gates are exercised throughout training. The comparison with the ungated dropout-aware model separates the training-time augmentation from the structural gate. Figure 3.2 illustrates the gating.

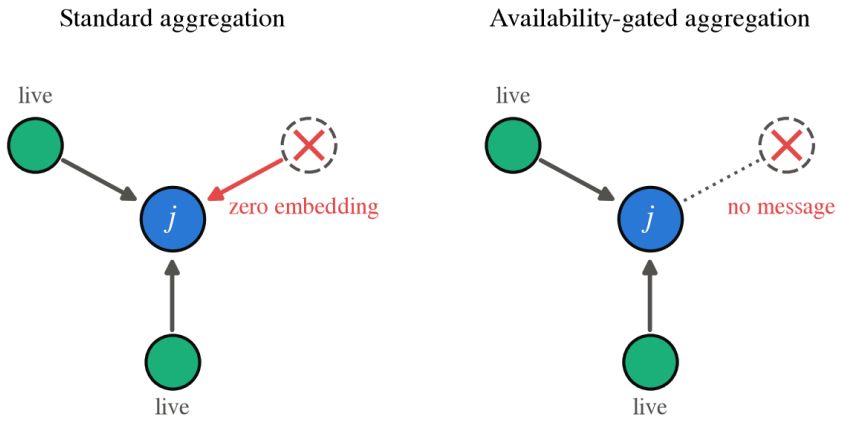


Figure 3.2: Availability-gated aggregation in SDR-STGNN.

3.6 Comparison and Stress-Test Methods

The thesis tests dropout-aware training against alternative remedies and against harder conditions on the same corpus, the same splits and the same frozen checkpoints. Table 3.4 lists the methods, and the following subsections describe them. Methods that need a physical model are described in Section 3.7.

Table 3.4: Comparison methods and stress tests of the sensor-loss study.

Method	What it varies relative to the random sweep
Neighbor-mean imputation	Fill of the dropped nodes before inference
Learned graph imputation	Fill produced by a trained graph autoencoder
Wider training grid (DA-wide)	Dropout levels seen in training
Targeted loss	Which sensors drop: those nearest the fault
Persistent loss	When the mask is redrawn: once per event
Clustered loss	How the dropped sensors are arranged: a connected block
Measurement noise	Accuracy of the sensors that still report
Classical baselines	Model family: no graph and no deep network

3.6.1 Imputation Before Inference

A natural alternative to training with missing data is to reconstruct the dropped sensors before inference and to present the result to a standard-trained model. The neighbor-mean imputer fills the three voltage channels of each dropped node with the per-time-step mean of its available one-hop neighbors in the undirected measured-only graph. If the whole neighborhood is dropped, the fill is zero. The mask of the imputed node is set to one, which is the most favorable setting for imputation because the model is told that the value is observed. The dropped nodes are selected exactly as in the zeroing sweep, so only the fill differs. The imputer is the training-free analogue of the graph imputation of Zhang et al. (2025).

The learned imputer is a masked autoencoder on the measured-only graph. A per-node encoder maps the flattened window and an availability flag to a vector of dimension 128 and concatenates a learned embedding of the node identity. Four residual layers with mean aggregation over the closed neighborhood follow, and a decoder returns the normalized voltages of the 20 time steps. It is trained on the training split to reconstruct the voltages of k randomly dropped nodes, with k drawn uniformly from 1 to 10 and with masks seeded independently of the evaluation. The optimizer is AdamW with a constant learning rate of 10^{-3}, the batch size is 64, and training runs for 60,000 steps. The checkpoint with the lowest validation reconstruction error is kept. One imputer is trained per seed and is paired with the standard-trained RGSAGE-Max of the same seed. The number of steps was decided by a rule fixed before the evaluation. An

initial run of 20,000 steps had a validation error that was still falling by more than 2% between steps 15,000 and 20,000, so the run was repeated with 60,000 steps. The imputer is used only with a standard-trained classifier, and joint training of imputer and classifier is not examined.

3.6.2 Harder Loss Patterns

Targeted loss drops the k sensors nearest to the fault. Proximity is the hop distance on the full 123-bus topology and not on the measured-only graph, because most fault buses are not nodes of the measured-only graph. The selection is a deterministic function of the label, so all models see the same mask. The no-fault windows have no fault bus and fall back to random selection. The reported score is therefore a conservative bound on the true worst case.

Persistent loss draws the dropped set once per event and holds it for all 40 windows of the event, as a real failure would behave. The set is a pure function of the event, the level k and the mask realization, derived through a cryptographic hash and not through a process-dependent hash, so it is the same for every model and batch size. Everything else is unchanged, so the difference to the random sweep isolates the persistence of the mask.

Clustered loss drops a connected block of the measured-only graph. The block is grown from a uniformly drawn root by repeatedly adding a uniformly drawn neighbor of the current block. It models sensors that share a communication path or a feeder segment. The policy changes how often each node is dropped as well as how the dropped nodes are arranged, so it does not isolate spatial correlation. The same limit applies to its comparison with the random sweep.

3.6.3 Measurement Noise

Zero-mean Gaussian noise is added to the voltage channels at inference. The standard deviation is specified in per-unit voltage and applied in normalized space as the physical value divided by the training standard deviation of the phase. The noise levels are 0.0005, 0.002 and 0.01 p.u. The largest level equals the one percent limit of the synchrophasor standard. Noise is added only where the mask is one, so unmeasured phases and dropped nodes receive none, and the mask channels are never perturbed. Noise and

sensor loss are applied together on a grid of the noise level and the number k of dropped sensors, on the frozen checkpoints and with the same masks as the dropout sweep.

3.6.4 Classical Baselines

To measure how much of the accuracy needs a graph model at all, a logistic regression and a random forest with 100 trees are trained on summary features. Each window is reduced, for every sensor and phase, to the minimum, mean, maximum and standard deviation over the 20 samples, and these are concatenated with the three mask values of the node. This gives 25 nodes with 15 values each, or 375 features per window. The minimum and the depth of the sag are the quantities that an impedance-style method uses, so the encoding is a fair one and not a straw man. The features are standardized, and the models are trained on a subsample of 93,696 training windows. The classifiers use the same splits and the same dropout masks as the neural models and emit the class of the window directly, so they need no soft vote.

3.7 The Physical Operator

3.7.1 Signature Table

The physical operator is a table of fault signatures computed with the same circuit simulator that generated the corpus. For every candidate location f, fault type τ, fault resistance R_{F} and load multiplier η, one power-flow solution gives the settled post-fault voltage magnitude of each phase at each of the 25 sensors. The table is written $\mathcal{S}[f, \tau, R_{\mathrm{F}}, \eta, i, p]$ in per-unit voltage. A second table $\mathcal{S}_0[\eta, i, p]$ holds the voltages of the healthy feeder at each load level, which gives the no-fault class a signature. Table 3.5 lists the axes.

The table is built along the path of the corpus. The feeder is compiled again for every solution, because the circuit simulator carries regulator tap positions from one solution to the next on a standing circuit, which was found to shift the signatures. The loads are set to η times the frozen pre-fault level of the corpus, which is 0.99953, since the corpus applies a small sinusoidal load modulation before the fault and never rescales the loads during the fault. Three solver steps are taken after fault injection so that the signature is

Table 3.5: Axes of the signature table.

Axis	Values
Location f	25 candidate fault buses, plus the no-fault hypothesis
Fault type τ	11 types, as in the corpus
Resistance R_{F}	18 values on a logarithmic grid from 0.05 to 30 Ω, with the training values 0.1, 1 and 10 Ω on the grid
Load multiplier η	17 values from 0.5 to 1.3 in steps of 0.05
Sensor and phase (i, p)	$25 \times 3 = 75$ channels, of which 55 are instrumented
Size	84,150 fault solutions and 17 healthy solutions; 3,366 hypotheses per class

the settled level. All 84,167 solutions converged. The resistance grid contains the three corpus resistances exactly, because a grid that left them off-grid let a nearby wrong resistance trade against a compensating load shift and corrupted the recovery of the load.

The table is the generator of the corpus run again, and not an approximation of it. A comparison on clean data therefore tests the fidelity of the operator and cannot show that the physical model beats the learned model, as noted in Section 1.8. The thesis therefore uses the operator under perturbations that the generator does not produce, namely sensor loss and measurement noise.

3.7.2 Observation and Residual

The residual compares a window with the signature of every hypothesis. The observation is the last time step of the window, converted back to per-unit voltage with the training statistics of the loader,

$$s_{ip} = x_{i,S,p}\,\mathrm{std}_p + \mathrm{mean}_p, \qquad \mathcal{A} = \{(i,p) : m_{ip} = 1\}. \tag{3.12}$$

The last step is the post-fault level whenever the window contains at least one post-fault sample, because the fault period is quasi-static. For window 20 it is the pre-fault level, and the residual correctly prefers the no-fault class. The observation uses no window index, no label and no event metadata. The set $\mathcal{A}$ contains the channels that are instrumented and not dropped, so dropped sensors are handled by construction: they simply do not enter the sum.

For each class the nuisance parameters are profiled out. The type, the resistance and

the load are not known, so the operator keeps the best-fitting hypothesis,

$$r_f(\mathcal{A}) = \min_{\tau, R_{\mathrm{F}}, \eta} \sum_{(i,p)\in\mathcal{A}} \frac{\big(s_{ip} - \mathcal{S}[f, \tau, R_{\mathrm{F}}, \eta, i, p]\big)^2}{\sigma_\varepsilon^2 + \sigma_0^2}, \qquad f = 1, \ldots, 25, \tag{3.13}$$

$$r_0(\mathcal{A}) = \min_{\eta} \sum_{(i,p)\in\mathcal{A}} \frac{\big(s_{ip} - \mathcal{S}_0[\eta, i, p]\big)^2}{\sigma_\varepsilon^2 + \sigma_0^2}. \tag{3.14}$$

Here σ_ε is a fixed floor and σ_0 is the level of the noise injected into the measurement, which is zero on the clean corpus and is set to the injected level under noise. The floor σ_ε is a single constant, chosen on the training split only so that the median squared residual per channel at the true class equals one. Its value is 7.2555×10^{-4} p.u. It is a numerical floor that comes from the discretization of the load axis and does not describe the accuracy of a physical instrument.

The raw residuals of wrong locations are very large, of order 10^5 to 10^6 with this floor, whereas the residual of the true location is of the order of the number of channels. The operator therefore shifts and halves them:

$$\rho_f = \tfrac{1}{2}\big(r_f - \min_g r_g\big) \;\geq 0. \tag{3.15}$$

The quantity ρ_f is in units of a Gaussian log-likelihood, so a difference of one is one nat of evidence. It is zero for exactly one class, and it does not depend on the number of available channels, which makes it comparable across dropout levels. The physics prior over the 26 classes is $P_{\mathrm{phys}} = \mathrm{softmax}(-\rho)$. Physics signature matching (PSM) is the training-free localizer that returns the class with the smallest residual, $\hat{y} = \arg\min_f \rho_f$. The margin $\rho_{(2)} - \rho_{(1)}$ between the best and the second-best class measures how decisive the evidence is.

The residual is computed in single precision by forming the difference before squaring. The expansion of the square into matrix products loses the true hypothesis to cancellation in single precision, so it is used only in double precision for the offline computation of the residual of the whole corpus. The residual is a function of the inputs and the fixed table, so it carries no gradient with respect to any learnable weight.

The table is checked against the corpus by four procedures. The bias of the table at the true hypothesis shows whether the build path diverged from the generator. The recovery of the load multiplier and the fault type at the true location, by profiling, tests

whether the load axis describes the corpus. The calibration of the floor checks that the median residual per channel at the true class is of order one. A reduction of the resistance grid to every second point checks that the grid is dense enough.

3.7.3 Identifiability

The table also measures how much the sensors can reveal in principle. For two locations f_1 and f_2, the separability is the closest approach of their two signature manifolds on the available channels,

$$D(f_1, f_2; \mathcal{A}) = \min_{(\tau_1, R_1, \eta_1), (\tau_2, R_2, \eta_2)} \sum_{(i,p) \in \mathcal{A}} \frac{\left(\mathcal{S}_{f_1} - \mathcal{S}_{f_2}\right)^2}{2(\sigma_\varepsilon^2 + \sigma_0^2)}, \tag{3.16}$$

which is in the same units as ρ. It is the residual that a perfect observation of f_1 would assign to f_2, and a value below one means that the two locations are indistinguishable at the floor of the operator. The thesis averages D over random sensor subsets for each k and reports the fraction of location pairs with $D < 1$ per resistance. The result describes the information content of the sensing infrastructure and does not depend on any trained model. It is compared with an empirical measure that uses no table: the nearest-neighbor confusion between training events at different locations, computed on the raw observations.

3.7.4 Physics-Based Imputation and Inference-Time Fusion

Two further uses of the operator need no training. In physics-based imputation, each dropped node is filled with the signature of the best hypothesis of PSM, profiled on the available channels only. The fill is normalized with the training statistics, repeated over the 20 time steps, and presented with a mask of one to the standard-trained RGSAGE-Max. The dropped nodes are those of the zeroing sweep, so the comparison with the neighbor-mean fill isolates the fill.

In inference-time fusion, the residual is combined with the logits of an existing trained model without retraining. The logits of every node are reduced by a multiple of the residual,

$$z'_{n,f} = z_{n,f} - \beta\, \rho_f(\mathcal{A}), \tag{3.17}$$

where the same ρ_f is used for every node n and $\beta \geq 0$ is chosen on validation data from the grid $\{0, 0.1, 0.25, 0.5, 1, 2, 4, 8, 16\}$. The result is then soft-voted as before. This is the cheapest test of whether the operator knows something that the network does not.

3.8 PC-STGNN: Physics-Consistent Fusion in the Network

3.8.1 Architecture

The physics-consistent spatio-temporal graph neural network, PC-STGNN, keeps the dropout-aware RGSAGE-Max backbone unchanged and fuses the residual into the logits of every node before the soft vote, as in Eq. (3.17), with β now a learned scalar initialized to zero. The fusion acts on the logits of each node, not on a graph-level score. Summing the softmax of the fused node logits gives

$$\sum_n \mathrm{softmax}(z'_n)_f \;\propto\; \sum_n \frac{\exp(z_{n,f})}{Z_n}\,\exp(-\beta\rho_f), \tag{3.18}$$

where Z_n is the normalizer of node n. Every node distribution is thus multiplied by the same physics prior before the votes are summed, which is a product of experts per node. The implementation acts on the node logits so that the node-level loss, the soft vote and the evaluation code stay unchanged. The physical path adds exactly one learned parameter, so the model has 95,259 parameters against 95,258 for the backbone, and the signature tables and the floor are fixed buffers that receive no gradient. Figure 3.3 shows the architecture.

3.8.2 Training Objective

Let z be the node logits of the backbone and z' the fused logits. The objective has three terms besides the main cross-entropy:

$$\mathcal{L}_{\mathrm{PC}} = \mathrm{CE}(z', y) + \lambda_1 \sum_f P(f|X)\,\rho_f(\mathcal{A}) + \lambda_2 \sum_f P(f|X)\,\rho_f(\mathcal{D}) + \gamma\,\mathrm{CE}(z, y). \tag{3.19}$$

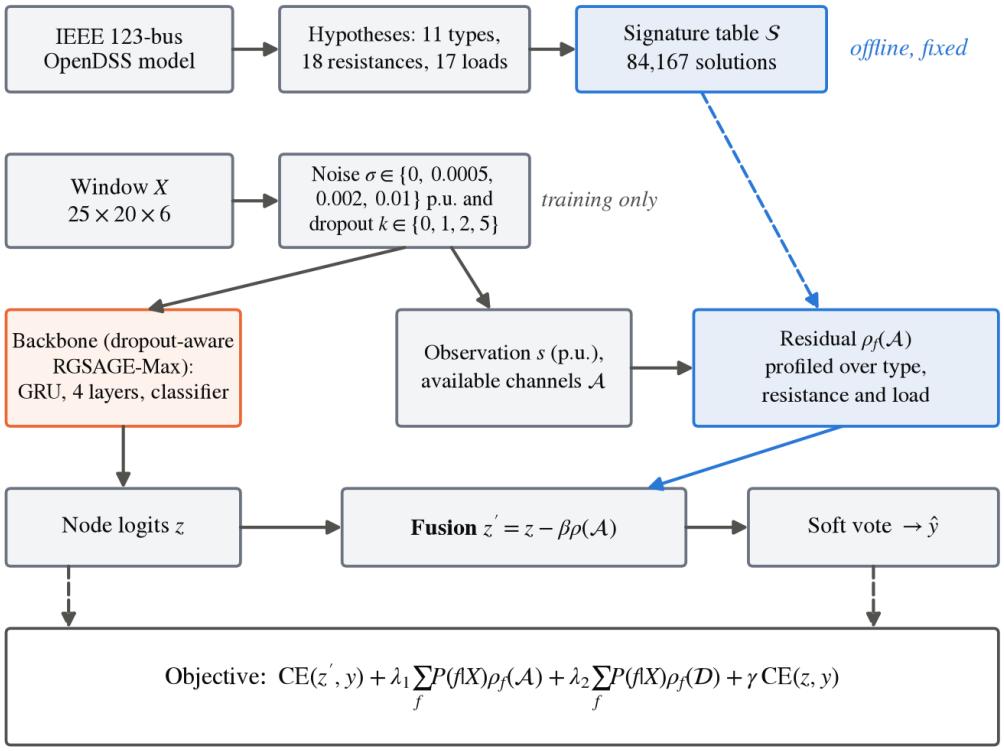


Figure 3.3: Architecture of PC-STGNN.

Here $P(f|X)$ is the normalized soft-vote distribution of the model. The second term is the expected residual on the available sensors. The third is a leave-k-out consistency term: the hypothesis that was profiled on the available sensors is scored on the sensors $\mathcal{D}$ that were dropped in training, whose true values are known at training time only. The profiling never sees $\mathcal{D}$, so the term is a consistency check and not a reconstruction loss. A reconstruction loss at the best hypothesis would have zero gradient, because the index of the best hypothesis is constant with respect to the weights. The two physical terms act on the network through $P(f|X)$ and not through the residual. The last term is a cross-entropy on the logits of the network alone, before fusion. Its weight is fixed at $\gamma = 1$ and is not tuned. Windows with no post-fault sample, window 20 of every fault event, are excluded from the two physical terms, because the physics correctly prefers the no-fault class there. This is the only use of the window index, and it applies in training only.

3.8.3 Noise Augmentation and Collapse Guard

On the noise-free corpus the residual at the true class is exactly zero, so the easiest solution for the fusion is to trust the physics completely and to let the network contribute nothing. Training under noise gives the physical term a finite and informative dynamic range. PC-STGNN is therefore trained with measurement noise on top of dropout-aware training. For every window, a noise level is drawn from $\{0, 0.0005, 0.002, 0.01\}$ p.u., with zero among the choices so that clean inputs remain in the data. Noise is added before the dropout is applied, the dropout levels are $\mathcal{K} = \{0, 1, 2, 5\}$ as before, and σ_0 in the residual is set to the level injected into each window. Noise changes neither the dropout masks nor the mask channels.

The auxiliary term of Eq. (3.19) and the noise are both needed to keep the network branch useful. A collapse guard monitors the network branch alone on the validation split at $k = 0$ without noise. An arm collapsed if the network-only macro-F1 is at most 0.10 or if more than 95% of its predictions are no fault. A collapsed arm is not eligible for selection, and a run that is collapsed at the end of epoch 3 is stopped.

3.8.4 Selection Rules, Reference Model and Verification

The weights λ_1 and λ_2 are chosen on the validation split by the macro-F1 averaged over $k \in \{0, 1, 2, 5, 10\}$ among the arms that pass the collapse guard, with one seed and 15 epochs per arm. The grid is $\lambda_1 \in \{0, 0.01, 0.0333, 0.1\}$ with $\lambda_2 = 0$, followed by one arm with $\lambda_2 = 0.1$ at the best λ_1. Within a run, the checkpoint is the epoch with the best validation macro-F1 at $k = 0$, as for all other models. Tuning runs never evaluate the test split.

The reference for every comparison is a model with the same recipe and no physics: the dropout-aware RGSAGE-Max with the same noise augmentation, called the noise-augmented reference model. Comparing PC-STGNN with a model trained without noise would confound noise augmentation with physics, so both models are trained with the same augmentation, for 30 epochs and at three seeds. Both are evaluated on the validation split on the grid of the noise levels $\{0, 0.0005, 0.002, 0.01\}$ p.u. and the sensor-loss levels $k \in \{0, 1, 2, 5, 10\}$, with three mask realizations.

Two checks guard the implementation before any training result is trusted. In the

null test, a PC-STGNN with $\lambda_1 = \lambda_2 = \beta = 0$ is trained from seed 42 and must reproduce the per-epoch loss and validation score of the dropout-aware backbone exactly. A second null test shows that $\gamma = 0$ reproduces the old objective and its gradients exactly on a fixed batch. In the gradient audit, finite differences on three deliberately chosen batches confirm that both physical terms send a gradient to the network weights and that the signature table receives none. The batches contain a window 20, a high dropout level and a batch with no dropped sensors. The last case is the most common in training and the one in which the leave-k-out term must vanish without producing undefined values.

3.9 Verification of the Evaluation Pipeline

The score at $k = 0$ of every degradation curve must equal the clean score recorded for the checkpoint, and the same command run twice gives identical results. For the imputation baselines, the sets of dropped nodes were compared with those of the zeroing sweep for every level and mask realization, and they are identical. An early version of the evaluator used the same mask seed in every batch, so the same subsets repeated. The seed is now derived from the batch index, and all dropout results were recomputed and not patched.

3.10 Summary

This chapter defined the corpus, the models, the protocol, the remedies and the physical operator. The next chapter reports the results.

Chapter 4

RESULTS AND INFERENCES

This chapter reports the measurements and what they imply. Unless stated otherwise, scores are the soft-voted macro-F1 on the frozen test split, given as mean $\pm$ standard deviation over the training seeds 42, 43 and 44, and every dropout result averages three mask realizations within each seed. Results of the physical operator that use the validation split are labeled as such, because the test split is not used for any decision or tuning.

4.1 Baseline Accuracy Under Full Observability

Table 4.1 gives the five baselines with all sensors reporting. The four spatio-temporal graph models clearly outperform the recurrent baseline, which shows that message passing matters when only six of the 25 fault locations carry a sensor. RGSAGE-Max is the most accurate model, with a macro-F1 of 0.846 ± 0.007. The graph-level score exceeds the node-level score for every model, so the soft vote improves on the prediction of any single node. Accuracy falls as the fault resistance rises: RGSAGE-Max drops from 0.949 at 0.1 Ω to 0.676 at 10 Ω, which agrees with the weak voltage signature of high-impedance faults. Fault detection is saturated, since every model reaches a no-fault recall of at least 0.9999 and a precision of at least 0.972, so the difficulty lies in localization. RGATv2 is the weakest graph model and the least stable across seeds. At the adopted depth of four layers, its variance suggests a training instability that was not addressed.

The choice of four message-passing layers is supported by a single-seed diagnostic on RGSAGE-Max, shown in Table 4.2. The score rises with every added layer, which confirms that a larger receptive field is the main driver of accuracy. The budgets of the diagnostic runs differ, as the table states, so the rows show the trend and are not a controlled comparison. The four-layer model with the full schedule of 30 epochs reaches 0.853 for this seed and shows no sign of over-smoothing. The three stable

Table 4.1: Baseline macro-F1 under full observability, mean ± standard deviation over three seeds. The best value per column is in bold.

	Macro-F1		Graph-level by resistance		
Model	Graph	Node	0.1 Ω	1 Ω	10 Ω
RGSAGE-Max	**0.846**±0.007	**0.718**±0.008	**0.949**	**0.893**	**0.676**
RGSAGE-Mean	0.786±0.023	0.660±0.025	0.906	0.843	0.587
RGCN	0.712±0.017	0.601±0.019	0.874	0.792	0.428
RGATv2	0.522±0.087	0.460±0.080	0.630	0.574	0.312
GRU	0.286±0.009	0.150±0.003	0.410	0.292	0.105

spatial models were still improving at the end of the epoch budget, since their best validation epoch was between 27.7 and 28.7 of 30, so their accuracies are a floor.

Table 4.2: Depth diagnostic for RGSAGE-Max, one seed (42), test macro-F1.

Message-passing layers	1	2	3
Epochs	30	15	15
Macro-F1	0.509	0.571	0.751
Message-passing layers	4	4	
Epochs	15	30	
Macro-F1	0.806	0.853	

4.2 Degradation Under Sensor Loss

Table 4.3 and Figure 4.1 show the standard-trained models under increasing sensor loss. Every model declines as sensors are removed, and no baseline keeps more than a third of its clean score at $k = 10$. The most accurate model degrades fastest. RGSAGE-Max falls from 0.846 to 0.048 and keeps 5.7% of its clean score, whereas RGCN keeps 23.2%. The accuracy ranking, RGSAGE-Max, RGSAGE-Mean, RGCN, RGATv2 and GRU, is essentially reversed by the robustness ranking.

The inversion does not depend on the normalization by the clean score. The absolute area under the curve places RGCN (0.432) above RGSAGE-Max (0.340), and the curves cross in absolute macro-F1. The relative lead of RGATv2 is an artifact of its low clean score. Its relative area of 0.626 falls to 0.324 in absolute terms, which is the fourth of five, so the thesis does not claim RGATv2 as robust. The comparison with the trivial reference of 0.0256 shows what collapse means: at $k = 10$ RGSAGE-Max reaches 0.048, only 1.9 times the score of a classifier that always answers no fault.

Table 4.3: Standard-trained models under sensor loss: macro-F1 at each level k, the retention at $k = 10$, and the relative and absolute AUDC, mean over three seeds.

Model	$k = 0$	$k = 1$	$k = 2$	$k = 5$	$k = 10$	Ret@10	AUDC (rel / abs)
RGSAGE-Max	0.846	0.736	0.590	0.236	0.048	5.7%	0.402 / 0.340
RGSAGE-Mean	0.786	0.720	0.647	0.393	0.108	13.8%	0.541 / 0.425
RGCN	0.712	0.664	0.610	0.418	0.164	23.2%	0.608 / 0.432
RGATv2	0.522	0.490	0.444	0.302	0.157	31.0%	0.626 / 0.324
GRU	0.286	0.273	0.252	0.111	0.051	17.9%	0.522 / 0.149

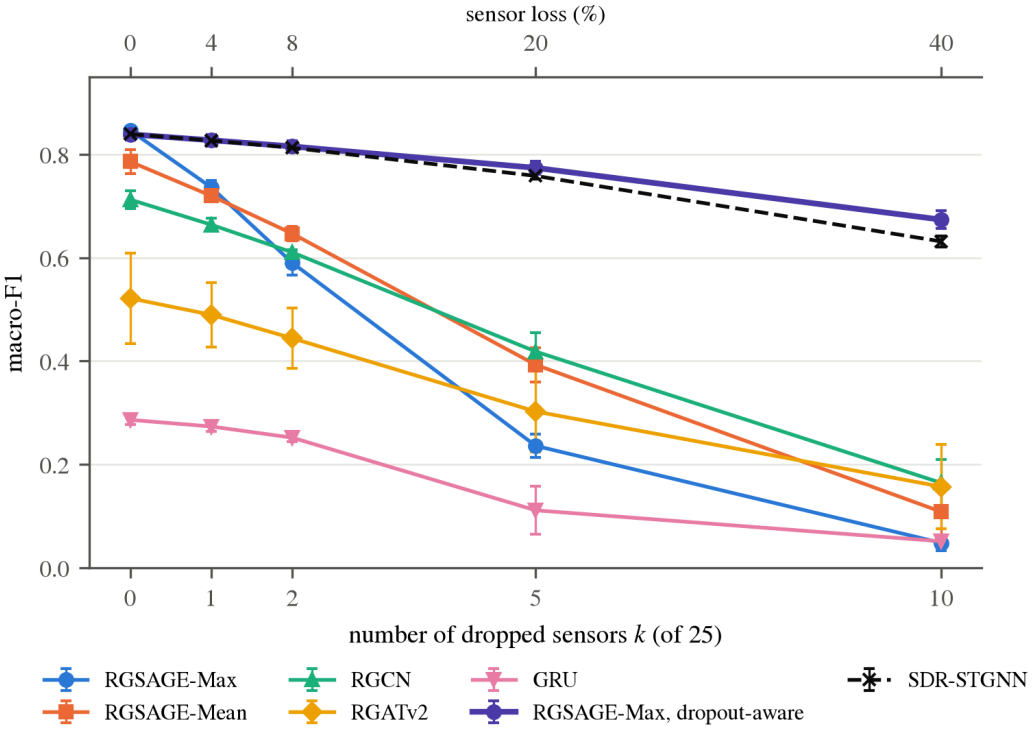


Figure 4.1: Degradation curves of the baselines and of the two mitigations.

The split of the score into detection and localization sharpens this picture. For the standard RGSAGE-Max at $k = 0$ the no-fault class has an F1 of 0.990 and the 25 fault classes have a mean F1 of 0.841. At $k = 10$ the no-fault F1 is still 0.201, but the localization score has fallen to 0.041. Sensor loss destroys localization before it destroys detection.

4.3 Dropout-Aware Training

Dropout-aware training changes the picture, as Table 4.4 shows. For RGSAGE-Max the macro-F1 at $k = 10$ rises from 0.048 to 0.674 ± 0.017, a gain of 0.626, and the

retention rises from 5.7% to 80.3%. The relative AUDC rises from 0.402 to 0.913. The clean score changes from 0.846 ± 0.007 to 0.839 ± 0.011, which is within one standard deviation. The gain holds for all five architectures, but the cost in clean accuracy is not the same for all of them. RGSAGE-Max loses less than 0.01 macro-F1 at $k = 0$, whereas RGCN drops by 0.099 and RGSAGE-Mean by 0.113. The statement that robustness comes without a cost in clean accuracy therefore holds for the RGSAGE-Max backbone and not for the whole family. The recurrent baseline also improves, but its retention remains limited to 25%, because it has no message passing with which to use the surviving sensors. Because retention is relative to the clean score, the relative leaders are again not the strongest in absolute terms: the dropout-aware RGSAGE-Max reaches 0.674 at $k = 10$, well above the 0.505 of RGATv2, whose retention is higher.

Table 4.4: Effect of dropout-aware training (DA) on all architectures: macro-F1 at $k = 0$ and $k = 10$, relative AUDC and retention at $k = 10$ for standard (Std) and dropout-aware training.

	$k = 0$		$k = 10$		AUDC		Ret@10 (%)	
Model	Std	DA	Std	DA	Std	DA	Std	DA
RGSAGE-Max	0.846	0.839	0.048	0.674	0.402	0.913	5.7	80.3
RGSAGE-Mean	0.786	0.673	0.108	0.550	0.541	0.922	13.8	81.9
RGCN	0.712	0.613	0.164	0.533	0.608	0.940	23.2	87.0
RGATv2	0.522	0.553	0.157	0.505	0.626	0.961	31.0	91.4
GRU	0.286	0.300	0.051	0.078	0.522	0.781	17.9	25.4

4.4 Comparison of Remedies

Table 4.5 compares all the ways of coping with sensor loss on the RGSAGE-Max backbone under the same masks. Replacing dropped sensors by the mean of their neighbors helps little: the relative AUDC rises from 0.402 to 0.476 and the retention at $k = 10$ from 5.7% to 11.8%. The reconstructed values still reach a model that was never trained to distrust them. A learned graph imputer reconstructs the dropped voltages with a root-mean-square error of 0.012 to 0.017 p.u. against 0.24 to 0.30 p.u. for the neighbor mean, a reduction by a factor of 18 to 20. Its classification gain is nevertheless modest (AUDC 0.570, retention 22.2%), and it does not survive correction for multiple tests over the neighbor-mean fill or over zeroing. The imputer did not fully converge for two of three seeds, since its validation loss was still changing by more than the 2% criterion at the

end of 60,000 steps. A fill from the physical operator, described in Section 3.7, performs better than the learned imputer (AUDC 0.601, retention 26.6%) but has a very large seed variance at high k, because a wrong best hypothesis under heavy loss produces a badly wrong fill. All three fills stay far below dropout-aware training.

Table 4.5: Remedies on the RGSAGE-Max backbone, macro-F1 at each level k, relative AUDC and retention at $k = 10$, mean over three seeds. Imputation is a no-operation at $k = 0$.

Variant	$k = 0$	$k = 1$	$k = 2$	$k = 5$	$k = 10$	AUDC	Ret@10
Standard (zeroing)	0.846	0.736	0.590	0.236	0.048	0.402	5.7%
Neighbor-mean imputation	0.846	0.739	0.637	0.336	0.099	0.476	11.8%
Learned graph imputation	0.846	0.766	0.686	0.447	0.187	0.570	22.2%
Physics-based imputation	0.846	0.775	0.701	0.479	0.225	0.601	26.6%
Dropout-aware training	0.839	0.827	0.815	0.774	0.674	0.913	80.3%
DA-wide ($k = 10$ in training)	0.833	0.823	0.812	0.776	0.704	0.928	84.5%
SDR-STGNN (gated)	0.839	0.827	0.813	0.758	0.631	0.892	75.2%

Adding the level $k = 10$ to the training grid (DA-wide) raises the macro-F1 at $k = 10$ by 0.030 and lowers the clean score by 0.006. Both differences have the same sign in all three seeds, but neither survives correction ($p_{\mathrm{Holm}} = 0.057$ for each), so the thesis keeps the original grid as the main method and reports the trade-off as a consistent observation. The availability-gated SDR-STGNN matches dropout-aware training but does not exceed it. Its relative area is 0.892 against 0.913, and its curve lies below the ungated curve at every level from $k = 1$ onward. At $k = 0$ the gate is inactive, so the two dropout-aware variants score identically (0.839). The gate is not free: it raises the single-event latency by 18% for no gain in robustness.

4.5 Stress Tests

4.5.1 Harder Loss Patterns

Table 4.6 compares the loss patterns for RGSAGE-Max. Targeted loss is far more damaging than random loss in the regime in which the model is still usable. With one sensor removed, the standard model falls to 0.448 ± 0.039 against 0.736 ± 0.013 under random loss, a drop from the clean score that is 3.6 times larger. At $k = 2$ the two policies give 0.361 and 0.590. Both differences have the same sign in all seeds but are not significant after correction ($p_{\mathrm{Holm}} = 0.052$ and 0.067). From $k = 5$ the two

policies agree, since the model has collapsed in either case. Dropout-aware training is penalized by targeting as well (AUDC 0.913 to 0.747, $p_{\text{Holm}} = 0.016$), so worst-case loss is a distinct stressor that the remedy does not absorb. The remedy still retains between 0.307 and 0.473 macro-F1 over the standard model under the same targeting ($p_{\text{Holm}} \leq 0.012$ at every level), and for $k = 5$ and $k = 10$ its worst-case score exceeds the average-case score of the standard model (0.600 against 0.236 and 0.521 against 0.048, $p_{\text{Holm}} = 0.006$ and 0.012). At $k = 1$ the two are indistinguishable ($+0.018$, $p_{\text{Holm}} = 0.333$).

Table 4.6: Loss patterns on RGSAGE-Max, macro-F1 at four levels and relative AUDC, standard (Std) and dropout-aware (DA) training.

Pattern	Training	$k = 1$	$k = 2$	$k = 5$	$k = 10$	AUDC
Random (reference)	Std	0.736	0.590	0.236	0.048	0.402
	DA	0.827	0.815	0.774	0.674	0.913
Targeted	Std	0.448	0.361	0.206	0.048	0.299
	DA	0.755	0.695	0.600	0.521	0.747
Persistent	Std	0.737	0.592	0.233	0.047	0.401
	DA	0.827	0.815	0.776	0.672	0.913
Clustered	Std	0.737	0.664	0.452	0.153	0.553
	DA	0.827	0.812	0.762	0.666	0.904

Persistent loss, in which the dropped set is held for the whole event, changes nothing. No point of either curve moves by more than 0.003, and no difference survives correction across the twelve paired comparisons (smallest $p_{\text{Holm}} = 0.46$). The reason is that the decision rule is per window: the soft vote pools the 25 nodes of one window and never the 40 windows of an event, so the prediction of a window depends only on its own mask, whose distribution is the same under both schemes. This equivalence would not hold for a decision rule at the level of the event.

Clustered loss is milder than scattered loss for the standard RGSAGE-Max, with an AUDC of 0.553 against 0.402 and a retention at $k = 10$ of 18.1% against 5.7%. Only the difference at $k = 5$ (0.452 against 0.236) survives correction ($p_{\text{Holm}} = 0.023$). For the dropout-aware model the two policies give nearly the same curve, with an AUDC of 0.904 against 0.913 and no difference above 0.012 macro-F1. The gain of the remedy over the standard model remains significant at every level under clustered loss ($p_{\text{Holm}} \leq 0.022$). The result is reported as an observation and not as an established effect, because a connected block does not drop every sensor equally often. At $k = 10$ the per-node

drop rate ranges from 0.18 to 0.87 against 0.40 under random loss, so the comparison does not isolate spatial correlation.

4.5.2 Measurement Noise

Noise costs real accuracy, and dropout-aware training does not protect against it (Table 4.7, Figure 4.2). For the standard RGSAGE-Max at $k = 0$ the macro-F1 falls from 0.846 to 0.786, 0.579 and 0.345 at noise levels of 0.0005, 0.002 and 0.01 p.u. The largest level is the one percent limit of the synchrophasor standard. Dropout-aware training is not better at any noise level at $k = 0$, and the differences from the standard model range from -0.008 to -0.034, with confidence intervals that include zero. The explanation follows from the mechanism of the remedy. Noise leaves the availability mask at one, so the learned response to a node that is dark has nothing to react to. The advantage of the remedy against sensor loss survives noise in attenuated form: at $k = 10$ the gain over the standard model is $+0.627$ without noise and $+0.579$, $+0.428$ and $+0.209$ with increasing noise. Noise and sensor loss compound below the trivial reference. The standard model at $k = 10$ and the largest noise level scores 0.018, below the 0.0256 of a classifier that always answers no fault.

Table 4.7: RGSAGE-Max under measurement noise and sensor loss, macro-F1 for standard (Std) and dropout-aware (DA) training.

	$k = 0$				$k = 10$			
σ (p.u.)	0	0.0005	0.002	0.01	0	0.0005	0.002	0.01
Std	0.846	0.786	0.579	0.345	0.048	0.040	0.032	0.018
DA	0.839	0.752	0.555	0.316	0.675	0.619	0.460	0.227

4.5.3 Unseen Fault Scenarios

The event-level split leaves every scenario in all three splits. On the split that holds out whole scenarios, accuracy is much lower, which is the largest limit on the generalization of the reported scores. The standard RGSAGE-Max reaches 0.490 ± 0.057 at $k = 0$, against 0.846 on the event-level split, and the dropout-aware model reaches 0.384 ± 0.030. Under sensor loss the standard model falls to 0.066 at $k = 10$, and the dropout-aware model keeps 0.265. The models therefore generalize to unseen load conditions

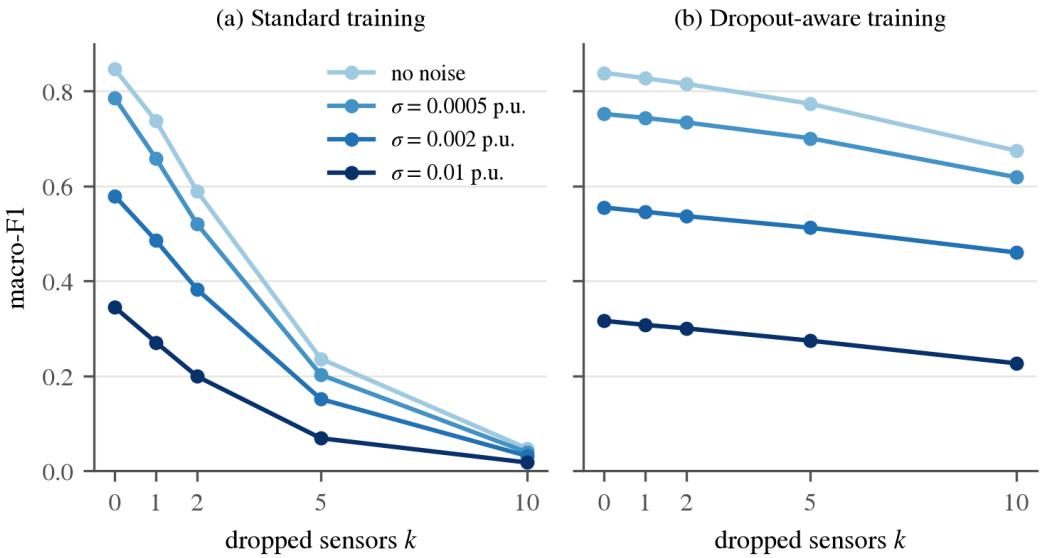


Figure 4.2: Effect of noise and sensor loss together on RGSAGE-Max.

of known fault configurations far better than to unseen configurations. This split was evaluated only for RGSAGE-Max, without statistical tests, so these numbers describe the size of the effect and not a tested ranking.

4.5.4 Classical Baselines and the Detection–Localization Split

A logistic regression on 375 summary features reaches a macro-F1 of 0.713 at $k = 0$ and a random forest reaches 0.633, both trained on 8% of the training windows and without any graph. The margin of RGSAGE-Max over the linear model is 0.133 and not the 0.560 that the recurrent baseline suggests. Under sensor loss the random forest is the most robust of the untreated models: it keeps 35.9% of its clean score at $k = 10$, with an absolute area of 0.470, above the 0.432 of RGCN. The linear model collapses to 0.039. The dropout-aware RGSAGE-Max, with a retention of 80.3% and an absolute area of 0.766, is more robust than any of them.

4.6 Statistical Significance

Table 4.8 reports the paired tests on the mitigation claims, over the three training seeds, with Holm correction across the seven tests of the family. Dropout-aware training improves the macro-F1 at $k = 10$ by 0.626, the retention by 74.6 percentage points and the area by 0.511, and all three survive correction ($p_{\text{Holm}} = 0.004$). It also ex-

ceeds neighbor-mean imputation by 0.574 ($p_{\mathrm{Holm}} = 0.010$) and the learned imputer by 0.486 ($p_{\mathrm{Holm}} = 0.037$). The area gain is significant for each of the five architectures ($p_{\mathrm{Holm}} \leq 0.027$), and the gain is significant at every level $k \geq 1$ ($p_{\mathrm{Holm}} \leq 0.042$). The two null claims of this thesis also test as null. The clean-accuracy cost of -0.008 has an interval that includes zero ($p_{\mathrm{Holm}} = 0.196$), and the advantage of dropout-aware training over SDR-STGNN in the area is also not significant ($p_{\mathrm{Holm}} = 0.196$), which is the statistical form of the statement that the gate matches but does not exceed it.

Table 4.8: Paired two-sided t-tests over three seeds on the RGSAGE-Max backbone ($\mathrm{df} = 2$). Δ is the mean seed-wise difference, and p_{Holm} is adjusted over the seven tests. Std is standard training and DA is dropout-aware training.

Comparison	Δ	95% CI	p_{Holm}
DA against Std, clean F1	-0.008	$[-0.024, +0.009]$	0.196
DA against Std, F1 at $k = 10$	$+0.626$	$[+0.557, +0.695]$	0.004
DA against Std, Ret@10	$+74.6$	$[+67.2, +82.0]$	0.004
DA against Std, AUDC	$+0.511$	$[+0.450, +0.571]$	0.004
DA against neighbor-mean imputation, F1 at $k = 10$	$+0.574$	$[+0.449, +0.699]$	0.010
DA against learned imputation, F1 at $k = 10$	$+0.486$	$[+0.253, +0.720]$	0.037
DA against SDR-STGNN, AUDC	$+0.021$	$[-0.010, +0.052]$	0.196

Two orderings of the baselines are not significant at three seeds. The clean ranking of the top pair, RGSAGE-Max over RGSAGE-Mean, has $p_{\mathrm{Holm}} = 0.085$ and no adjacent pair survives. The accuracy–robustness inversion, in which RGCN exceeds RGSAGE-Max at $k = 10$ in every seed by a factor of 2.4 to 6.7 ($+0.116$), has $p_{\mathrm{Holm}} = 0.429$ because the size of the gap varies from seed to seed. A bootstrap over the 6,175 test events gives an interval for the top pair about five times narrower than the seed-level interval, so the number of training seeds limits these orderings, and they are reported as consistent observations. The spread between mask realizations within a seed (0.0002 to 0.0046 macro-F1) is small against the spread between seeds (0.008 to 0.061).

4.7 Computational Cost

Dropout-aware training changes only the data. The standard and the dropout-aware RGSAGE-Max have the same 95,258 parameters and the same single-event latency (0.639 against 0.640 ms), and both process 6,038 windows per second. The only extra

cost is 2.7% in training time per epoch. SDR-STGNN adds no parameters but raises the latency by 18% (0.755 against 0.639 ms) for no robustness gain. The activation memory is 145 MiB for all models, because the shared encoder dominates it. A training run of RGSAGE-Max takes about 6.7 hours on the NVIDIA T1000 for standard training and 6.9 hours for dropout-aware training, and the runs of the five architectures take between 4.7 and 12.3 hours.

4.8 The Physical Operator

4.8.1 Fidelity of the Signature Table

Table 4.9 lists the checks of Section 3.7. The median bias of the table against the corpus at the true hypothesis is 1.3×10^{-3} p.u. Profiling the load and the type at the true location recovers the load multiplier within one grid step for 97.3% of the 28,875 training events and the fault type for 90.9%. The residual floor, rescaled by one global constant chosen on the training split, gives a median squared residual per channel of 1.00 at the true class. Halving the resistance grid changes the localization macro-F1 on the 25 fault classes by -0.0013, within noise. The first of these checks is a construction check, because the table is the generator of the corpus run again, so the bias can only measure whether the build path diverged. A build with a standing circuit that kept regulator taps from one solution to the next disagreed with the corpus by more than 10^{-3} p.u. on 43.3% of 1,000 tested events, against 1.8% when the circuit was compiled again for every solution. The table was therefore built the second way. The remaining disagreements are 6.4 to 7.3×10^{-3} p.u., one regulator tap step, and 16 of the 18 cases lie at 10 Ω.

Table 4.9: Checks of the signature table on the training split.

Check	Criterion	Measured
Median bias at the true hypothesis	Discretization scale	1.31×10^{-3} p.u.
Fault type recovered by profiling	$> 90\%$	90.9%
Load recovered within one step	$> 90\%$	97.3%
Median residual per channel at true class	Between 0.3 and 3	1.00
Effect of halving the resistance grid	Below 0.01 macro-F1	-0.0013

4.8.2 Identifiability

The separability of Eq. (3.16) shows that 25 magnitude-only sensors carry enough information to separate almost every pair of fault locations. Of the 325 pairs among the 26 classes, 1.5% are indistinguishable at the floor of the operator for $k = 0$, 2 and 5, and 1.8% for $k = 10$. The most confusable pair is the same at every level, and its separability falls from 0.0074 to 0.0047. The limit is a property of high resistance and not of the number of sensors: at 0.1 and 1 Ω no pair is indistinguishable at any level, and at 10 Ω the fraction is 0.6% at $k = 0$ and 2 and 0.9% at $k = 5$ and 10. Figure 4.3 shows the separability maps. The table agrees with the empirical confusability of the raw observations: a nearest-neighbor classifier on the 55 instrumented channels reaches an accuracy of 88.9%, and the rank correlation between the separability of a pair and its empirical confusion is -0.655 with $p = 4 \times 10^{-38}$ over 300 location pairs. No pair that the table calls confusable is separable in the corpus. The 51 pairs that the table calls separable but the raw observations confuse are expected, because the nearest-neighbor measure pools all types, resistances and loads and does not profile them.

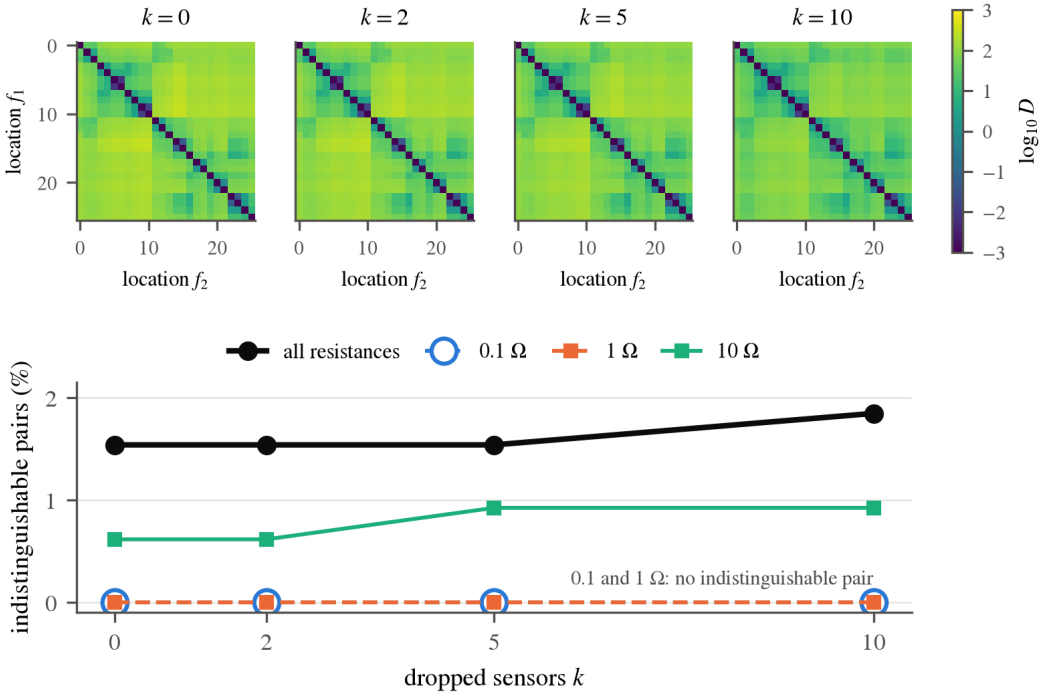


Figure 4.3: Identifiability of the fault location from 25 magnitude-only sensors.

4.8.3 Physics-Only Baselines

Table 4.10 orders the training-free methods on the validation split, with the same 26-class window-level macro-F1 as the networks. A rank-one sag template without profiling scores 0.162 at every level and is almost uninformative on its own. Matching the nearest signature at the nominal load, without profiling, scores 0.649 at $k = 0$. Profiling the type, the resistance and the load (PSM) adds 0.123 and scores 0.772. The score of PSM is nearly flat under sensor loss, since the residual simply drops the lost channels from its sum, and it falls only from 0.772 to 0.749 at $k = 10$. In distribution, the learned model is better: the dropout-aware RGSAGE-Max reaches 0.855 on the validation split at $k = 0$ for the same seed. The physical operator therefore does not win on its own ground. Its value lies in the perturbed conditions, as the next subsection shows. The score of PSM on clean data is a check of the fidelity of the operator and not a competitor of the learned model, since the table and the corpus come from the same simulator.

Table 4.10: Training-free methods on the validation split, macro-F1 at each level k and the absolute area.

Method	$k = 0$	$k = 1$	$k = 2$	$k = 5$	$k = 10$	AUDC (abs)
Rank-one template	0.162	0.162	0.162	0.162	0.162	–
Nearest signature, no profiling	0.649	0.644	0.640	0.628	0.601	0.626
PSM	0.772	0.771	0.769	0.763	0.749	0.762

4.8.4 Inference-Time Fusion

Combining the residual with an existing dropout-aware checkpoint at inference, as in Eq. (3.17), raises the score in every condition tested (Table 4.11). The gain is small on clean data ($+0.039$) and large where the network has the least to work with: $+0.166$ at $k = 10$, $+0.086$ for faults of 10 Ω, and up to $+0.213$ at the noise level of 0.01 p.u. This is the condition against which dropout-aware training has no effect. The weight β that the validation selects rises with the noise level, which means that the physics does more correction work where the mask-based mechanism has nothing to react to. These are preliminary results: they use one seed, a subsample of 30,000 validation windows, and a weight chosen on the same data on which it is scored, and the fusion was applied to a checkpoint that was never trained to expect it.

Table 4.11: Inference-time fusion on the dropout-aware RGSAGE-Max, seed 42, validation subsample of 30,000 windows.

Condition	Selected β	F1 at $\beta = 0$	F1 with fusion	Change
Clean, $k = 0$	0.1	0.858	0.897	$+0.039$
Clean, $k = 0$, faults of 10 Ω	0.1	0.667	0.753	$+0.086$
Sensor loss, $k = 10$	0.1	0.695	0.860	$+0.166$
Noise 0.0005 p.u.	0.1	0.762	0.799	$+0.037$
Noise 0.002 p.u.	16	0.592	0.745	$+0.153$
Noise 0.01 p.u.	2	0.325	0.538	$+0.213$

The cost of computing the residual is not negligible. Online, it takes 44.7 ms for a batch of 32 on the T1000, nine times more than the budget set for it, and it is limited by computation and not by memory. For the offline evaluation the residual of the whole corpus was therefore computed once and stored for all dropout levels and mask realizations. The stored values agree with the online computation to a relative error of 1.6×10^{-5} and select the same hypothesis in all cases.

4.8.5 Screening of Three First-Order Physics Grafts

Before the operator was built, three ways of adding first-order physical knowledge to the RGSAGE-Max backbone were screened, each with at most one extra parameter. The first scaled the messages by the impedance of the edge between two sensors, the second added a bias from a shared-path sensitivity to the logits, and the third added a loss that penalized the mismatch of the sag pattern. At seed 42 and with 15 epochs, none improved on the backbone (Table 4.12). The two learned scalars show how the model treated the priors. The weight of the first prior was driven to zero (-17.4 before the softplus) and removed it, and the sign of the second was reversed (-1.90). The baseline in Table 4.12 is the validation score of the same backbone at epoch 15. The third, run at the full recipe and three seeds, matches the baseline in clean accuracy (0.847 ± 0.018 against 0.846 ± 0.007) and with dropout-aware training (0.837 against 0.839), and it makes training less stable, with a seed variance 2.6 times larger. Its apparent advantage against noise in the single-seed screen reversed at three seeds (-0.032 at the largest noise level). The physics available in voltage magnitudes alone is weak when it is injected in these ways, which motivated the stronger operator of the preceding sections.

Table 4.12: Screening of three physics grafts, RGSAGE-Max, seed 42, 15 epochs.

Variant	Validation F1	Test F1	Learned scalar
Backbone (epoch 15)	0.816	–	–
Impedance-weighted messages	0.807	0.801	Weight driven to zero
Sensitivity bias on the logits	0.811	0.809	Sign reversed (-1.90)
Sag-pattern loss	0.811	0.806	None

4.9 PC-STGNN

4.9.1 Verification

The checks of Section 3.8 passed. With $\lambda_1 = \lambda_2 = \beta = 0$ the model reproduced all 30 epochs of the dropout-aware backbone exactly, and the training loss, the validation scores and the best validation score (0.8555) were identical to the last digit. With $\gamma = 0$ the new objective gave the same loss and identical gradients for all 17 parameter tensors on a fixed batch. The finite-difference audit passed on all three batches: both physical terms sent a non-zero gradient to the network weights, the weight β received a gradient, the signature table received none, and the leave-k-out term vanished without undefined values when no sensor was dropped.

4.9.2 Why Noise Augmentation and the Auxiliary Loss Are Needed

Trained without noise, all six tuning arms reached a degenerate solution. Measured on the 248,000 validation windows at $k = 0$, the network branch alone predicted no fault for 99.9 to 100% of the windows and scored 0.0256, the trivial reference. On fault windows the fused prediction agreed with PSM for between 87.7% and 98.0% of the windows, and for 95.9% in the best arm, whose fused accuracy matched that of PSM (85.6% against 85.8%). The whole gain over PSM came from the no-fault windows, where the fused accuracy was 87.0% against 49.8%. The model had become physics matching with a learned no-fault bias, as the design analysis had predicted: when the residual at the true class is exactly zero, trusting it completely is the easiest solution.

Noise augmentation reduced the problem but did not remove it. The share of no-fault predictions of the network branch fell to about 55%, close to the true 50%, but its macro-F1 stayed at 0.078 without the physical loss and 0.061 with it, below the criterion

of 0.10. The auxiliary loss on the network-only logits resolved it. After five epochs, with $\gamma = 1$, the network branch alone reached a macro-F1 of 0.571, 7.3 times the value without the auxiliary term, and the fused score stayed at 0.879. The auxiliary term is therefore fixed at $\gamma = 1$ and is part of the model.

4.9.3 Selection of the Loss Weights

Table 4.13 shows the five arms of the final tuning, all with noise and the auxiliary loss, at one seed and 15 epochs. All arms pass the collapse guard. The loss λ_1 hurts monotonically: the averaged validation score falls from 0.878 at $\lambda_1 = 0$ to 0.818 at $\lambda_1 = 0.1$, and the loss shows up in the no-fault windows, where the fused accuracy falls from 100.0% to 74.2% because a larger β lets the physics override correct no-fault decisions. The expected residual on the available sensors therefore does not contribute. The held-out term with $\lambda_2 = 0.1$ is neutral to slightly positive. The best validation score averaged over k is 0.8797, which selects $\lambda_1 = 0$ and $\lambda_2 = 0.1$. The margin over $\lambda_1 = \lambda_2 = 0$ is 0.0019 and is not a tested difference. Even at the end of this tuning, the fused prediction on fault windows agrees with PSM for 91.5 to 94.7% of the windows, so at clean $k = 0$ the localization is still largely physical. The question that matters is the behavior under noise and sensor loss, which is the comparison of the next subsection.

Table 4.13: Final tuning of the loss weights, seed 42, 15 epochs, validation split. The columns give the best epoch, the fused macro-F1 at $k = 0$ and averaged over k, and the macro-F1 of the network branch alone.

λ_1	λ_2	Best epoch	Fused F1, $k = 0$	Fused F1, mean over k	Network-only F1
0	0	10	0.893	0.878	0.738
0.01	0	12	0.890	0.878	0.788
0.0333	0	12	0.858	0.852	0.717
0.1	0	14	0.823	0.818	0.741
0	0.1	12	0.894	0.880	0.757

4.9.4 Comparison With the Noise-Augmented Reference

The comparison that decides whether the learned fusion helps is between PC-STGNN and the noise-augmented reference model. Both are trained for 30 epochs at three seeds and evaluated on the validation split under noise and sensor loss, as defined in

Section 3.8. Table 4.14 reserves the cells for these values. They are placeholders and will be filled when the runs are complete.

Table 4.14: Comparison of PC-STGNN with the noise-augmented reference, validation macro-F1, mean over three seeds. The values are not yet available.

Condition	Reference	PC-STGNN	Difference
Clean, $k = 0$	[value pending]	[value pending]	[value pending]
Noise 0.002 p.u., $k = 0$	[value pending]	[value pending]	[value pending]
Noise 0.01 p.u., $k = 0$	[value pending]	[value pending]	[value pending]
Clean, $k = 10$	[value pending]	[value pending]	[value pending]
Noise 0.01 p.u., $k = 10$	[value pending]	[value pending]	[value pending]

4.10 Discussion

Four findings stand out. First, clean accuracy is an unreliable guide to robustness. The model that is best with all sensors is the most fragile when they fail, in absolute as well as relative terms, although the ordering of the middle models is indicative only at three seeds. The max aggregation forwards the largest activation of a neighborhood, so one corrupted message can dominate a live node, whereas the normalized sum and the mean average it away.

Second, the remedy works because it changes what the mask means. In the clean corpus the mask channels are static: they identify the phase instrumentation of a unit and never change, so a standard model reads the mask as a fixed fingerprint of the node and not as a live signal. A dropped sensor shows the all-zero pattern, which that node never shows in training, so the model reads the zero voltages as a valid near-zero measurement. Dropout-aware training makes the mask vary within the history of a node, so the all-zero pattern acquires the meaning that the neighbor is dark. This account fits three observations. An explicit gate is redundant, because the model has learned what the gate hard-codes. Plausible numbers do not replace the training signal, because the deficit is misplaced trust and not a lack of values. And the gain is smallest for the recurrent baseline, which has no path for information to route around a dark node. The same account explains why noise defeats the remedy: noise leaves the mask at one.

Third, the conclusions have limits. Scores on the event-level split fall by about

0.36 on unseen scenarios for the standard model, worst-case loss and noise are stressors that the remedy does not absorb, and the data are quasi-static magnitudes from one simulated feeder.

Fourth, the physical operator is informative but does not replace the learned model. Fusion at inference helps most under noise and heavy sensor loss in the preliminary test, whereas injection through the loss was neutral or harmful, and a stable trained fusion needed noise augmentation and an auxiliary loss. Whether the trained fusion beats the noise-augmented reference is the question that Table 4.14 will answer.

4.11 Summary

The most accurate model is the most fragile, and dropout-aware training restores 80.3% of the clean score at 40% sensor loss at no inference cost. Neither a gate nor imputation matches it.

Chapter 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Summary of the Work

The thesis studied fault localization in a distribution grid in which only a few sensors are installed, and it asked what happens to a learned localizer when sensors fail or are noisy after deployment. It adopted the framework of Karabulut et al. (2026) on the IEEE 123-bus feeder, regenerated a corpus of 1.65 million windows, and benchmarked five architectures at three seeds. It then measured their degradation under sensor loss at inference, developed dropout-aware training as a remedy, and compared the remedy with an availability-gated architecture and with three kinds of imputation. The conclusions were tested against worst-case, persistent and clustered loss, noise, a wider training grid, a classical baseline and a scenario-held-out split. In the second part, a table of simulated fault signatures defined a fixed physical operator, used to measure identifiability, to localize without training, to impute and to fuse with a trained model. A physics-consistent model with one added parameter was designed, verified and tuned.

5.2 Conclusions

The conclusions follow the research questions of Section 1.6.

RQ1: degradation and its relation to accuracy. Localization degrades sharply with sensor loss. The most accurate baseline, RGSAGE-Max, falls from 0.846 to 0.048 when 10 of 25 sensors are lost and keeps only 5.7% of its clean score, which is 1.9 times the score of a classifier that always answers no fault. Clean accuracy is an unreliable guide to robustness: the accuracy ranking of the five baselines is essentially reversed by the robustness ranking, in absolute as well as relative terms. Sensor loss destroys localization before detection.

RQ2: remedies. Dropout-aware training raises the macro-F1 of RGSAGE-Max at 40% sensor loss from 0.048 to 0.674 and the retention from 5.7% to 80.3% ($p_{\mathrm{Holm}} =$

0.004), at no cost in parameters or latency and with 2.7% more training time per epoch. It improves the area under the retention curve for all five architectures, although for RGCN and RGSAGE-Mean it costs about 0.1 macro-F1 under full observability. The availability-gated model matches the remedy but does not exceed it, and it adds 18% to the latency. Neighbor-mean, learned and physics-based imputation recover little, because a model that was never trained to distrust the filled values still trusts them. Robustness is therefore better obtained during training than through the architecture or at the input.

RQ3: harder conditions. The advantage of the remedy persists under persistent and clustered loss, but two conditions limit it. Targeted loss of the sensors nearest the fault is a distinct stressor that the remedy does not absorb, since its area falls from 0.913 to 0.747, although the remedy still outperforms the standard model under the same targeting. Measurement noise defeats the remedy completely at full observability, because noise leaves the availability mask unchanged and the learned response to a dark node has nothing to react to. On the scenario-held-out split the standard model falls from 0.846 to 0.490, so the reported accuracy holds for unseen loads and not for unseen fault configurations.

RQ4: information content of the sensors. Twenty-five magnitude-only sensors carry enough information to separate almost every pair of fault locations: 1.5% of the pairs are indistinguishable for $k = 0$ to 5 and 1.8% for $k = 10$, and the limit lies at high resistance and not in the number of sensors. The physical operator does not beat the learned model on its own ground, since training-free matching reaches 0.772 against 0.855 for the learned model on the validation split. It is nearly insensitive to sensor loss and, applied at inference to a trained checkpoint, improved the score in every condition tested in a preliminary test, most strongly under noise and heavy sensor loss.

RQ5: fusion in the network. The fusion can be added without adding capacity or leaking labels: it adds one parameter, reduces exactly to the backbone when switched off, and sends gradient to the network weights and not to the fixed table. Its design needed two further elements. Without noise augmentation the network learns to defer entirely to the physics, and without an auxiliary loss on the network-only logits it still fails to localize on its own. The loss on the expected residual of the available sensors did not contribute, and the held-out term was neutral to slightly positive. Whether the

trained fusion outperforms a model trained with the same noise augmentation and no physics is decided by Table 4.14. [Conclusion on this comparison pending completion of the runs.]

5.3 Limitations

The limitations are those of the scope in Section 1.8. All data come from simulation of one feeder with a fixed sensor placement and quasi-static RMS magnitudes, so field performance and transient content are not tested. The statistics rest on three seeds, so several orderings are only observations. The clustered comparison does not isolate spatial correlation, because a connected block drops some sensors much more often than others. The learned imputer did not fully converge for two seeds and was used only with a standard-trained classifier. The signature table comes from the simulator of the corpus, so clean-data comparisons favor the physical operator. The scenario-held-out split covers RGSAGE-Max only, and the depth of four layers was selected on it and applied to every architecture.

5.4 Future Enhancements

Several extensions follow from the work. The evaluation of the physics-consistent model is to be extended by the ablations that test whether the gain comes from physics: a shuffled assignment of signatures to locations, a learned decoder of matched size, and a wider backbone. The model is to be evaluated on resistances outside the training values, with a test set that was generated and sealed in advance, and under a perturbed signature table, to measure its tolerance to an imperfect feeder model. A sensor that keeps reporting wrong values can be studied with a per-sensor trust score derived from the residual. Other directions are the validation on additional feeders such as the IEEE 34-bus system and on reconfigured topologies, the use of voltage phase angles, which the simulator computes but the thesis does not use, and joint training of an imputer and a classifier. The clustered comparison can use equal drop rates, the depth can be chosen per architecture, more seeds would test the observed orderings, and field μPMU data would test the findings beyond simulation.

REFERENCES

1. Asiri, A. and Somasundaram, A. (2025). "Graph convolution network for fraud detection in bitcoin transactions." *Scientific Reports*, 15, 11076.

2. Brody, S., Alon, U., and Yahav, E. (2022). "How attentive are graph attention networks?." *International Conference on Learning Representations (ICLR)*.

3. Chen, K., Hu, J., Zhang, Y., Yu, Z., and He, J. (2020). "Fault location in power distribution systems via deep graph convolutional networks." *IEEE Journal on Selected Areas in Communications*, 38(1), 119–131.

4. Dogoulis, P., Tit, K., and Cordy, M. (2025). "KCLNet: physics-informed power flow prediction via constraints projections." *arXiv preprint arXiv:2506.12902*.

5. Fan, M., Xia, J., Zhang, H., and Zhang, X. (2024). "Fault location method of distribution network based on VGAE-GraphSAGE." *Processes*, 12(10), 2179.

6. Hamilton, W., Ying, Z., and Leskovec, J. (2017). "Inductive representation learning on large graphs." *Advances in Neural Information Processing Systems*, Vol. 30. 1024–1034.

7. Jayapal, S. and Annamalai, R. (2025). "Graph neural network with Hessian-based locally linear embedding for cancer metastasis analysis in lymph nodes using DeepLab segmentation." *IEEE Access*, 13, 46448–46458.

8. Karabulut, B., Manna, C., and Develder, C. (2025). "Generalization of graph neural network models for distribution grid fault detection." *arXiv preprint arXiv:2510.03571*.

9. Karabulut, B., Manna, C., and Develder, C. (2026). "Robustness of spatio-temporal graph neural networks for fault location in partially observable distribution grids." *arXiv preprint arXiv:2604.20403*.

10. Kipf, T. N. and Welling, M. (2016). "Semi-supervised classification with graph convolutional networks." *arXiv preprint arXiv:1609.02907*.

11. Kirthan, P., Anbazhagan, M., Reddy, R., and Kurunathan, H. (2026). "Towards uncertainty-calibrated traffic flow prediction using combinatorial graph neural networks." *Proceedings of the 12th International Conference on Vehicle Technology and Intelligent Transport Systems (VEHITS)*. 79–91.

12. Krishnathevar, R. and Ngu, E. E. (2012). "Generalized impedance-based fault location for distribution systems." *IEEE Transactions on Power Delivery*, 27(1), 449–451.

13. Li, W. and Deka, D. (2021). "PPGN: physics-preserved graph networks for real-time fault location in distribution systems with limited observation and labels." *arXiv preprint arXiv:2107.02275*.

14. Liao, W., Bak-Jensen, B., Pillai, J. R., Wang, Y., and Wang, Y. (2021). "A review of graph neural networks and their applications in power systems." *arXiv preprint arXiv:2101.10025*.

15. Liu, F., Xie, L., Yu, K., Wang, Y., Zeng, X., Bi, L., et al. (2023). "A novel fault location method based on traveling wave for multi-branch distribution network." *Electric Power Systems Research*, 224, 109753.

16. Livani, H., Evrenosoglu, C. Y., and Centeno, V. A. (2013). "A machine learning-based faulty line identification for smart distribution network." *North American Power Symposium (NAPS)*. 1–6.

17. Ngo, Q.-H., Nguyen, B. L. H., Zhang, J., Schoder, K., Ginn, H., and Vu, T. (2025). "Deep graph neural network for fault detection and identification in distribution systems." *Electric Power Systems Research*, 247, 111721.

18. Nguyen, B. L. H., Vu, T., Nguyen, T.-T., Panwar, M., and Hovsapian, R. (2022). "1-D convolutional graph convolutional networks for fault detection in distributed energy systems." *IEEE 1st Industrial Electronics Society Annual On-Line Conference (ONCON)*. 1–6.

19. Nyangon, J. (2026). "Engineering hybrid physics-informed neural networks for next-generation electricity systems: a state-of-the-art review." *arXiv preprint arXiv:2605.21903*.

20. Okoyomon, E., Yaniv, A., and Goebel, C. (2025). "Physics-informed inductive biases for voltage prediction in distribution grids." *arXiv preprint arXiv:2509.25158*.

21. Peng, C., Xia, F., Saikrishna, V., and Liu, H. (2022). "Physics-informed graph learning." *arXiv preprint arXiv:2202.10679*.

22. Rong, Y., Huang, W., Xu, T., and Huang, J. (2020). "DropEdge: towards deep graph convolutional networks on node classification." *International Conference on Learning Representations (ICLR)*.

23. Rossi, E., Kenlay, H., Gorinova, M. I., Chamberlain, B. P., Dong, X., and Bronstein, M. (2021). "On the unreasonable effectiveness of feature propagation in learning on graphs with missing node features." *arXiv preprint arXiv:2111.12128*.

24. Sachin, S., Narayanan, V. S., Trisha, M. S., Reddy, K. H., and Mahadevan, A. (2025). "AGNet: an adaptive encoder-decoder GNN for spatio-temporal forecasting." *Proceedings of the International Conference on AI and Robotics (AIR), Lecture Notes in Networks and Systems*. 131–149.

25. Sangameswar, P. K. and Anbazhagan, M. (2026). "Dropout-aware training for sensor-robust fault localization with spatio-temporal graph neural networks." *International Conference on Machine Learning and Data Engineering (ICMLDE 2026), Procedia Computer Science, accepted*.

26. Zhang, L., Hua, D., Ji, T., Qian, T., and Wang, J. (2025). "Fault location in power distribution networks with massive missing data: a graph-based imputation and contrastive learning approach." *IEEE Transactions on Automation Science and Engineering*, 22, 16825–16837.